RESEARCH-ARTICLE

A Lightweight PUF With Immunity to Machine Learning Attacks Based on a Weak-PUF-Assisted Reconfigurable LFSR and Temporal Feedback

ZHIYUAN CHEN, Anhui University, Hefei, Anhui, China

LIANG YAO, Anhui University, Hefei, Anhui, China

XIANGYU DONG, Anhui University, Hefei, Anhui, China

JINLONG LEI, Anhui University, Hefei, Anhui, China

LANGYU HE, Anhui University, Hefei, Anhui, China

YAN JIN, Anhui University, Hefei, Anhui, China

View all

Published: 09 March 2026
Accepted: 01 March 2026
Revised: 10 February 2026
Received: 06 January 2025

A Lightweight PUF With Immunity to Machine Learning Attacks Based on a Weak-PUF-Assisted Reconfigurable LFSR and Temporal Feedback

ZHIYUAN CHEN, School of Integrated Circuits, Anhui University, Hefei, China
LIANG YAO*, School of Integrated Circuits, Anhui University, Hefei, China
XIANGYU DONG, School of Integrated Circuits, Anhui University, Hefei, China
JINLONG LEI, School of Integrated Circuits, Anhui University, Hefei, China
LANGYU HE, Anhui University, Hefei, China
YAN JIN, Anhui University, Hefei, China
YUNLAI ZHU, Anhui University, Hefei, China
YING ZHANG, Anhui University, Hefei, China
XIUMIN XU, School of Integrated Circuits, Anhui University, Hefei, China
YINGCHUN LU, School of Microelectronics, Hefei University of Technology, China, hefei, China
ZHENGFENG HUANG, Hefei University of Technology, Hefei, China

Strong Physical Unclonable Functions (PUFs) are critical for lightweight authentication in the Internet of Things (IoT). However, traditional Strong PUF designs are susceptible to advanced Machine Learning (ML) modeling attacks. In this paper, we propose a novel modeling-attack-resilient Strong PUF architecture that transforms the challenge-response mapping from a statically approximable function into a mathematically rigorous Keyed Pseudo-Random Function (Keyed-PRF). The proposed architecture features two core innovations: a Sponge-Based Configuration Mechanism (SCM) that utilizes reliable Weak PUFs to dynamically configure the feedback polynomial of a Linear Feedback Shift Register (LFSR), effectively creating a device-specific secret key; and a Response-Modulated State Evolution Mechanism (RM-SEM), where the instantaneous physical responses of the underlying Arbiter PUF determine the evolution step size of the LFSR. This creates a deep temporal feedback loop that transforms the system into a Hidden Markov Model (HMM), blocking gradient-based learning strategies. We also introduce a reliability screening strategy based on a strict bit-error-rate threshold to ensure stability. Experimental results on Xilinx Artix-7 FPGAs demonstrate that the proposed PUF maintains a prediction accuracy of approximately 50% against four mainstream modeling attacks even with 1 million training Challenge-Response Pairs (CRPs). Furthermore, the

*Corresponding author.

This work was supported in part by the National Natural Science Foundation of China (NSFC) Nos. 62027815, 62174048, 62274052. Zhiyuan Chen and Liang Yao contributed equally to this work.
Authors' Contact Information: Zhiyuan Chen, School of Integrated Circuits, Anhui University, Hefei, Anhui, China; e-mail: 674697201@qq.com; Liang Yao, School of Integrated Circuits, Anhui University, Hefei, Anhui, China; e-mail: liangyao@mail.hfut.edu.cn; Xiangyu Dong, School of Integrated Circuits, Anhui University, Hefei, Anhui, China; e-mail: wb2214122@stu.ahu.edu.cn; Jinlong Lei, School of Integrated Circuits, Anhui University, Hefei, Anhui, China; e-mail: C02114220@stu.ahu.edu.cn; Langyu He, Anhui University, Hefei, Anhui, China; e-mail: 1203361351@qq.com; Yan Jin, Anhui University, Hefei, Anhui, China; e-mail: 3632422051@qq.com; Yunlai Zhu, Anhui University, Hefei, Anhui, China; e-mail: zhuyunlai@ahu.edu.cn; Ying Zhang, Anhui University, Hefei, Anhui, China; e-mail: yingz@ahu.edu.cn; Xiumin Xu, School of Integrated Circuits, Anhui University, Hefei, Anhui, China; e-mail: xiuminxu@126.com; Yingchun Lu, School of Microelectronics, Hefei University of Technology, China, hefei, anhui Province, China; e-mail: luyingchun@hfut.edu.cn; Zhengfeng Huang, Hefei University of Technology, Hefei, China; e-mail: huangzhengfeng@139.com.

ACM 1557-7309/2026/3-ART
https://doi.org/10.1145/3801570

design exhibits excellent uniformity, uniqueness, and reliability, achieving these security properties with minimal hardware overhead.

Additional Key Words and Phrases: Physical Unclonable Function, Machine Learning, Linear Feedback Shift Register, Obfuscation

1 Introduction

In the era of the Internet of Everything (IoE), Internet of Things (IoT) devices have permeated every aspect of modern life. However, this ubiquity introduces unique and formidable security challenges [24]. The majority of IoT nodes operate under strict constraints regarding power budget, silicon area, and cost, rendering them incapable of supporting complex traditional cryptographic protocols. Furthermore, as these devices are increasingly exposed to uncontrolled physical environments, existing authentication methods are highly susceptible to forgery and spoofing attacks. Consequently, there is an urgent demand within the hardware security community for a lightweight security primitive capable of establishing a Root of Trust (RoT), without incurring prohibitive overheads [2].

Physical Unclonable Functions (PUFs) have emerged as a promising solution to address these challenges. By leveraging the congenital parametric mismatches—the unavoidable and uncontrollable random variations inherent in nanoscale manufacturing processes—PUFs extract unique digital fingerprints for integrated circuits [3, 4]. Among various architectures, Strong PUFs are particularly distinguished by their exponentially large Challenge-Response Pair (CRP) space [12]. This massive entropy content affords Strong PUFs a distinct ascendency in authentication protocols, as it is computationally infeasible for an adversary to exhaustively enumerate all possible responses within a realistic timeframe, thereby enabling lightweight authentication without reliance on heavy computational resources.

Nevertheless, the security of Strong PUFs is not impeccable. Research indicates that due to the reliance on cascaded stages or repetitive structural units, the relationship between challenges and responses often exhibits a strong linear additive nature [2]. This predictable mathematical structure serves as a vulnerability exploited by Machine Learning (ML) modeling attacks [16, 21]. An adversary does not need to access the internal circuitry; by collecting a subset of CRPs as a training dataset and employing ML algorithms, they can construct a high-precision numerical model. Once trained, this model can accurately predict the supposedly unpredictable responses, effectively achieving a mathematical cloning of the target device with high accuracy [6, 13, 26]. This vulnerability precipitates a severe trust crisis for existing Strong PUF architectures in the face of intelligent attacks.

To conceal the linear correlation between challenges and responses in APUF, the academic community has proposed nonlinear structures such as XOR APUF [22], IPUF [14], and MPUF [18]. However, these designs typically rely on simply stacking nonlinear modules or introducing additional obfuscation logic, which precipitates a significant surge in hardware area and power consumption, rendering them incompatible with the strict lightweight constraints of IoT devices [11]. Furthermore, studies indicate that these variants remain vulnerable to advanced machine learning modeling attacks [5, 13, 16, 19, 20].

Delving into the root causes of the failure of traditional obfuscation schemes reveals that mere engineering complexity cannot fundamentally alter the inherent mathematical properties of PUFs. In this regard, Ganji et al. [6] provide a rigorous framework, proposing that PUF security be scrutinized through the lens of Boolean Analysis. Within this framework, traditional designs often exhibit low Average Sensitivity. This metric characterizes the dependency between PUF outputs and input bits, implying that despite obfuscation, the underlying mapping may degenerate into a so-called junta function (where the output is determined by only a small subset of input variables). Furthermore, following the methodology in [6], the concept of Noise Sensitivity quantifies the deviation from the Strict Avalanche Criterion (SAC). Analysis shows that low noise sensitivity results in the concentration

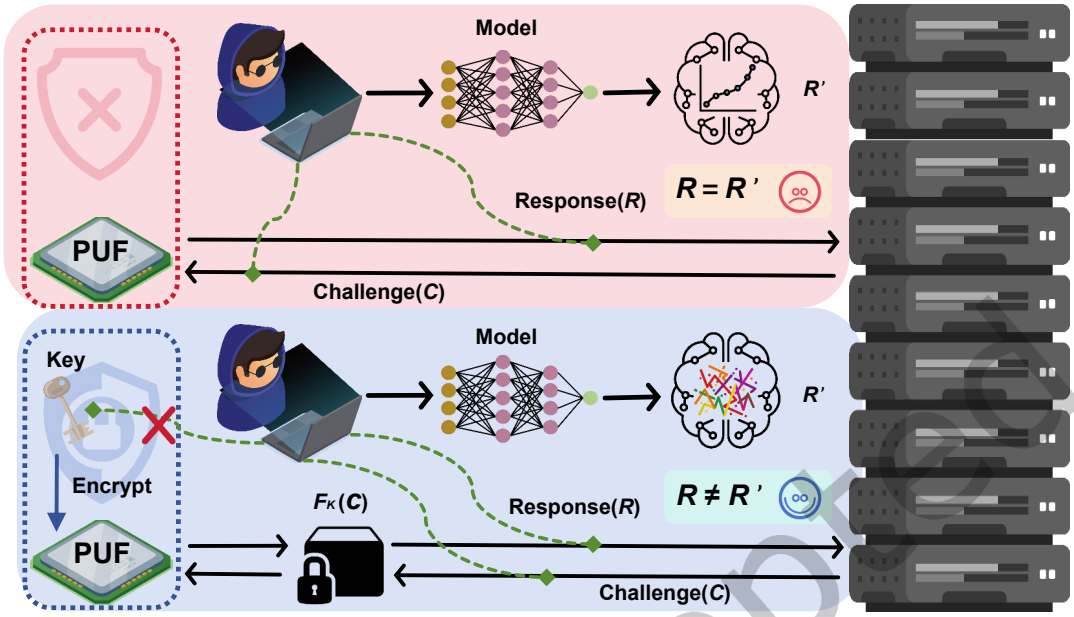

Fig. 1. Comparison between traditional structural obfuscation and the proposed Keyed-PRF paradigm. (a) Traditional designs based on $R = f_{phy}(C)$ are vulnerable to model fitting ($R \approx R'$). (b) The proposed Keyed-PRF architecture transforms the mapping into $R = F_K(C)$, effectively severing the gradient-based learning path ($R \neq R'$).

of spectral energy on the low-frequency coefficients of the Fourier spectrum. Such mathematical smoothness enables low-degree algorithms to construct high-precision models in polynomial time. This implies that the purported security of many seemingly robust designs may stem not from theoretical unlearnability, but rather from the practical limitation that attackers have not yet collected a dataset satisfying the required polynomial sample complexity.

To fundamentally mitigate the modeling threat, we argue for a paradigm shift: instead of relying exclusively on structural obfuscation, the design must incorporate a secret key to elevate the physical mapping into a mathematically rigorous Keyed Pseudo-Random Function (Keyed-PRF). It is important to clarify that, although the entropy source of a Strong PUF originates from truly random manufacturing variations, the input-output mapping of a fabricated device remains static and deterministic. Therefore, in the context of modeling resistance, a secure Strong PUF should be behaviorally indistinguishable from a PRF. Our design aims to transform the physical mapping from a statically approximable model $R = f_{\text{phy}}(C)$ into a cryptographic construct $R = F_K(C)$, as illustrated in Figure 1. This approach elevates the security requirement from the relatively low bar of function approximation to the cryptographic hardness of distinguishing a pseudorandom function from a Random Oracle. Consequently, this theoretically renders gradient-based learning strategies ineffective; the resulting pseudorandom output landscape denies the optimizer valid gradient information, making the attack complexity largely independent of the training set size.

Implementing this theoretical Keyed-PRF paradigm in hardware raises a critical practical question: how should the secret key K be instantiated and managed? Weak PUFs play an indispensable role. As detailed in [2], distinct from Strong PUFs that target an expansive CRP space, Weak PUFs (e.g., SRAM or Butterfly PUFs)

specialize in leveraging intrinsic process variations to extract a unique, high-entropy digital fingerprint. Their paramount advantage lies in the Key-on-Demand capability: the secret key K is derived in real-time from the physical microstructure upon power-up and vanishes immediately upon power-off. This volatile mechanism leaves no persistent traces within the circuitry, effectively eliminating the security risks associated with static storage. Furthermore, given their negligible logic resource consumption, Weak PUFs serve as the ideal anchor for constructing tamper-resistant and lightweight Keyed-PRF architectures.

Notably, the strategy of leveraging Weak PUFs as an entropy source to fortify Strong PUF designs has been actively explored in recent literature. These pioneering works empirically demonstrate the potential of such hybrid architectures in mitigating modeling attacks, providing valuable practical evidence for the viability of key-based defense mechanisms. However, while these studies represent significant strides in security validation, they exhibit certain limitations:

- **Prohibitive Hardware Overhead.** Many existing designs fail to achieve a favorable trade-off between security complexity and resource efficiency. For instance, the cSOI PUF proposed by Xu et al. [28] introduces a complex obfuscation mechanism to protect the delay network. As detailed in their architecture, this design requires a dedicated Challenge Obfuscation Block (COB) array containing a True Random Number Generator (TRNG) and four weak PUF instances for every pair of challenge bits. This architecture leads to a hardware consumption that scales significantly with the bit-width. Similarly, the SCD-PUF by Peng et al. [15] employs a multi-stage obfuscation scheme incorporating chaotic maps and shuffling algorithms. The hardware implementation of the Logistic chaotic map necessitates fixed-point multiplication and iterative arithmetic operations, while the Knuth-Durstenfeld shuffle algorithm imposes complex control logic requirements. These arithmetic-heavy and logic-intensive modules result in a substantial area penalty.
- **Inadequate Weak PUF Reliability.** The stability requirements for the secret key K are fundamentally different from and far more stringent than traditional PUF metrics. Due to the strict Avalanche Effect inherent in cryptographic functions, a flip of even a single bit in the key K triggers a drastic change (approx. 50%) in the output R, leading to immediate authentication failure. However, existing hybrid architectures often fail to meet this rigorous standard. A clear example of this misconception is found in the RPFO PUF [10], where the authors reported that 95.7% of the Weak PUF cells achieved a reliability greater than 99%, deeming this range acceptable. However, even assuming the best-case scenario where all cells reach 99% reliability, the probability of generating a stable 128-bit key is merely $0.99^{128} \approx 27.6\%$. This implies that the system is statistically destined to fail in over 70% of its operation attempts. More critically, other recent works such as the SW PUF by Tang et al. [23] and the CryptoPUF by Gao et al. [7] integrate complex cryptographic primitives or logic that mandate a stable seed, yet they merely assert high reliability without implementing or discussing rigorous error-correction or stabilization mechanisms.

Based on the above analysis, the motivations behind our design can be summarized into the following three points. First, since the avalanche effect of key errors leads to system failure, absolute key stability is mandatory. To guarantee this, we explicitly adopt a strict Margin-Based Stress Testing Strategy that identifies Weak PUF cells capable of withstanding artificially induced loads, thereby ensuring 100% stability. Second, we propose a holistic lightweight architecture where the Sponge-Based Configuration Mechanism, utilizing an auxiliary Linear Feedback Shift Register (LFSR), avoids complex initialization logic, while the subsequent state evolution relies on a larger LFSR with simple sequential logic. Third, possessing a secret key is insufficient if the underlying function remains mathematically smooth; therefore, we implement a Response-Modulated State Evolution Mechanism (RM-SEM) to disrupt this differentiability. By coupling state updates with instantaneous physical responses, we transform the system into a Hidden Markov Model (HMM), preventing attackers from acquiring valid gradient information.

In summary, the main contributions of this paper are as follows:

- A lightweight Keyed-PRF design is instantiated by rigorously stress-tested Weak PUFs. This ensures that the dynamic configuration logic is both strictly unpredictable and absolutely stable.
- The proposed architecture features a resource-efficient design where both the configuration phase (via Sponge construction) and the obfuscation phase (via RM-SEM) are optimized for minimal area overhead, significantly outperforming state-of-the-art complex obfuscation schemes.
- The RM-SEM is introduced to transform the challenge-response mapping into a state-dependent HMM process. This mechanism theoretically breaks the differentiability required by ML, rendering the design resilient against advanced modeling attacks.
- To enhance operational reliability, a response-based pre-screening method is developed for the overall Strong PUF. By filtering out challenges sensitive to noise, the operational reliability of the final obfuscated responses is significantly improved.
- Comprehensive evaluations are conducted using both simulation frameworks and Xilinx Artix-7 FPGA implementations. The results confirm that the design passes standard cryptographic statistical tests and achieves approximately 50% prediction accuracy against against four mainstream modeling attacks, validating its security.

The remainder of this paper is organized as follows: Section 2 introduces the preliminaries. Section 3 details the proposed configuration and obfuscation process. Section 4 provides a theoretical security analysis. Section 5 presents the simulation results regarding statistical metrics, reliability screening, and learnability assessment. Section 6 details the FPGA implementation, validating the resilience against modeling attacks. Finally, Section 7 concludes the paper.

2 Preliminaries

This section introduces the fundamental concepts and mathematical models required for the subsequent discussions. First, we detail the circuit structure and the linear delay model of the Arbiter PUF (APUF). Second, the operating principles of the Configurable Linear Feedback Shift Register (CLFSR) are presented. Third, we define the key quality metrics used to evaluate PUF performance. Finally, we outline the machine learning modeling attack algorithms targeting Strong PUFs.

2.1 Arbiter PUF

As illustrated in Figure 2, an n-bit APUF consists of n cascaded switch blocks and a terminal arbiter. Its operation is rigorously characterized by a linear cumulative delay model. Let Δ denote the total delay difference between the upper and lower paths, expressed as the inner product $\Delta = \Phi \cdot \Omega^\top$. Here, $\Phi = [\phi_0, \ldots, \phi_n]$ represents the parity-check feature vector derived from the challenge vector $C = [c_0, \ldots, c_{n-1}]$. Specifically, the elements of Φ are defined as $\phi_j = \prod_{k=j}^{n-1}(1 - 2c_k)$ for $0 \le j < n$, and the final element is fixed as $\phi_n = 1$. The vector $\Omega = [\omega_0, \ldots, \omega_n]$ signifies the physical delay parameters, where the coefficients are recursively calculated as $\omega_0 = \alpha_0$, $\omega_n = \beta_{n-1}$, and $\omega_i = \alpha_i + \beta_{i-1}$ for $i \in [1, \ldots, n-1]$. Crucially, these components are determined by the internal switch unit delays (p_i, q_i, r_i, s_i) shown in Figure 2, formulated as $\alpha_i = (p_i - q_i - r_i + s_i)/2$ and $\beta_i = (p_i - q_i + r_i - s_i)/2$. Finally, the arbiter digitizes Δ into a binary response r:

$$r = \begin{cases} 1, & \text{if } \Delta > 0, \\ 0, & \text{if } \Delta < 0. \end{cases} \tag{1}$$

This linear model reveals that the APUF response r is determined by a hyperplane partition in the feature space defined by $\Phi \cdot \Omega^\top = 0$. This geometric property constitutes the fundamental reason for its inherent vulnerability to linear modeling attacks.

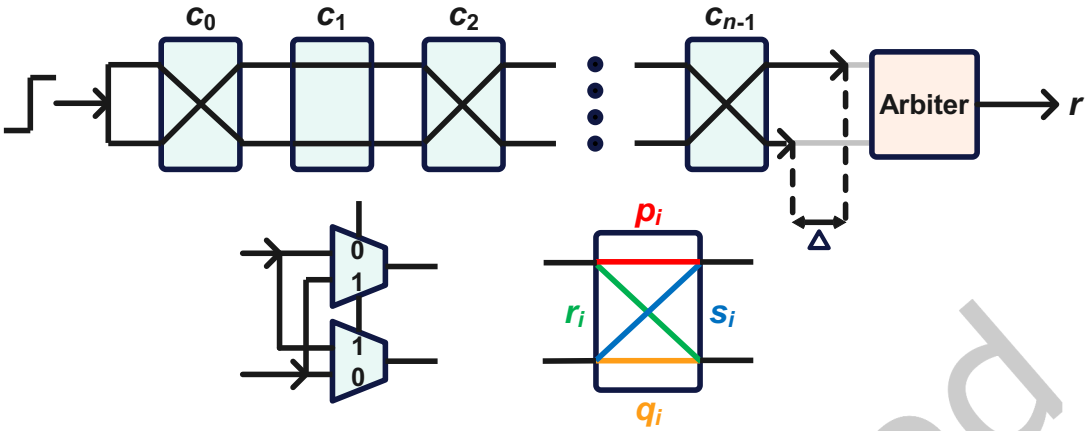

Fig. 2. Schematic diagram of an n-stage Arbiter PUF structure.

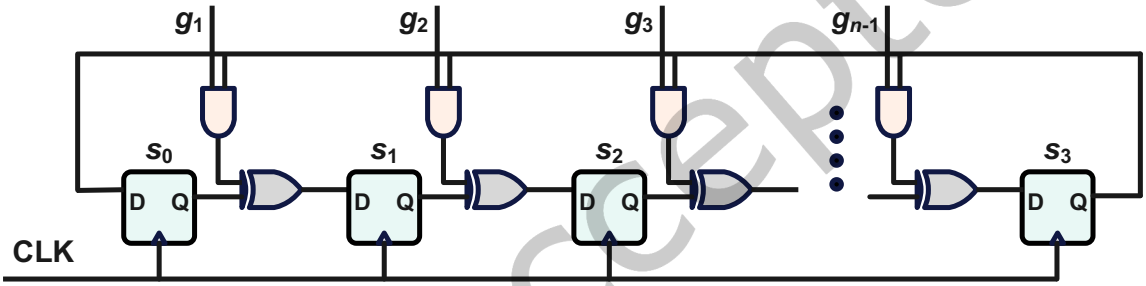

Fig. 3. Architecture of the n-stage Configurable Galois LFSR.

2.2 Configurable Galois LFSR

To balance hardware efficiency with polynomial flexibility, we employ a CLFSR based on the Galois topology. Unlike standard fixed-tap LFSRs, the proposed architecture integrates a switch array in its feedback path, controlled by a configuration vector G (derived from the system parameter G_{poly}). As illustrated in Figure 3, let $G = [g_1, \ldots, g_{n-1}]^\top$ denote the generator coefficient vector. The physical connectivity of the i-th feedback tap is strictly governed by the component g_i. Specifically, $g_i = 1$ activates the feedback path (closing the switch), while $g_i = 0$ physically disconnects it. Thus, G dynamically defines the characteristic polynomial of the circuit. The state update mechanism is modeled as a linear transformation over GF(2). Let $S^t = [s_0^t, \ldots, s_{n-1}^t]^\top$ represent the state vector at cycle t. The recursive relation is given by:

$$S^t = \mathbf{M} \cdot S^{t-1} \tag{2}$$

Here, $\mathbf{M} \in \{0,1\}^{n \times n}$ is the state transition matrix. Crucially, $\mathbf{M}$ is configured by G as follows:

$$\mathbf{M} = \begin{pmatrix} 0 & 0 & \cdots & 0 & 1 \\ 1 & 0 & \cdots & 0 & g_1 \\ \vdots & \ddots & \ddots & \vdots & \vdots \\ 0 & \cdots & 1 & 0 & g_{n-2} \\ 0 & 0 & \cdots & 1 & g_{n-1} \end{pmatrix} \tag{3}$$

As formulated in Eq. (3), the last column of $\mathbf{M}$ is populated by the coefficients of G, enabling run-time reconfiguration of the entropy generation logic.

2.3 Quality Metrics

Uniformity measures the distribution balance of '0's and '1's in the PUF responses. It is essentially equivalent to the normalized Hamming Weight (HW) of the response vector. For an ideal PUF, the probability of generating a '1' or '0' should be equal. Given an n-bit response sequence R, the uniformity is calculated as:

$$\text{Uniformity} = \frac{1}{n}\sum_{i=1}^{n} r_i \times 100\% \tag{4}$$

where r_i denotes the i-th bit of the response. The ideal value for uniformity is 50%.

Uniqueness reflects the capability of the PUF to distinguish between different devices. It is evaluated by calculating the average Inter-chip Hamming Distance (HD) of responses generated by different chips under the same set of challenges. For a set of N chips, the uniqueness is defined as:

$$\text{Uniqueness} = \frac{2}{N(N-1)}\sum_{u=1}^{N-1}\sum_{v=u+1}^{N}\frac{\text{HD}(R_u, R_v)}{n} \times 100\% \tag{5}$$

where R_u and R_v denote the response vectors of the u-th and v-th chips, respectively. The ideal uniqueness value is 50%.

Reliability indicates the stability of PUF responses under varying environmental conditions, such as temperature and voltage fluctuations. It is assessed by the Intra-chip HD, which measures the difference between responses reproduced by the same chip. Let R_{ref} be the reference response and R_t be the response sampled at the t-th measurement (or under specific conditions). The reliability is expressed as:

$$\text{Reliability} = 100\% - \frac{1}{m}\sum_{t=1}^{m}\frac{\text{HD}(R_{ref}, R_t)}{n} \times 100\% \tag{6}$$

where m is the total number of measurements. The ideal reliability value is 100%.

2.4 Machine Learning Attack Algorithms

Logistic Regression (LR) is a supervised statistical learning method widely used for binary classification [17]. LR applies the Sigmoid function, $y = 1/(1 + e^{-z})$, to map the linear delay sum $z = w \cdot \Phi$ into a probability value within $(0, 1)$. The weight vector w, representing the physical delays, is iteratively optimized by minimizing the log-likelihood loss function via gradient descent. Since the fundamental physical model of an APUF is determined by a linear hyperplane in the feature space, LR can efficiently converge to this boundary, making it the most effective attack against linear PUF architectures.

CMA-ES is a derivative-free, population-based evolutionary algorithm designed for difficult non-linear optimization problems [8]. It simulates biological evolution by maintaining a multivariate normal distribution of the model parameters. In each generation, a population of offspring (candidate delay vectors) is sampled. The parameters (mean and covariance matrix) of the distribution are updated based on the fitness (prediction accuracy) of the most successful offspring. Unlike LR, CMA-ES does not rely on gradient information. This makes it particularly powerful against obfuscated PUF architectures where the objective function landscape is rough, discontinuous, or non-differentiable.

The Multilayer Perceptron (MLP) is a classical feed-forward Artificial Neural Network (ANN) consisting of an input layer, one or more hidden layers, and an output layer [21]. MLP utilizes non-linear activation functions (e.g., ReLU or Tanh) within its neurons to capture complex data patterns. The network weights are trained using the

backpropagation algorithm to minimize prediction error. According to the Universal Approximation Theorem, an MLP with sufficient hidden units can approximate any continuous function. This capability allows MLP to learn the non-linear decision boundaries formed by simple structural obfuscations, such as XORing multiple APUF responses.

Deep Neural Networks (DNN) represent the cutting edge of modeling attacks, employing deeper architectures to model highly complex non-linearities. As detailed by Wisiol et al. [26], DNNs leverage heuristic initialization and advanced optimizers (e.g., Adam) to approximate the latent delay values of individual component PUFs within a composite structure. DNNs have demonstrated the ability to break security parameters previously thought safe against LR and MLP. They can effectively disentangle the complex parity functions in large-scale XOR-APUFs or Interpose-PUFs, posing the most severe threat to modern Strong PUF designs.

3 Proposed PUF Architecture

This section elucidates the design of the proposed modeling-attack-resilient Strong PUF. First, we introduce the preparatory Sponge-Based Configurable Phase, elaborating on how the system performs initialization and achieves configuration of its internal logic using Weak PUFs. Second, the Obfuscation Process is described in detail, explaining the complex non-linear transformations applied to the challenges and responses. To facilitate a concrete understanding of this abstract procedure, a algorithm and a simplified numerical walk-through example are provided. Finally, we present the Weak PUF Module, along with the specific reliability enhancement methods adopted to ensure stability.

3.1 Sponge-Based Initialization and Configuration

To transform raw physical entropy E_{raw} into a configuration stream G characterized by high complexity and global dependency, we propose the Sponge-Based Configuration Mechanism implemented via a compact Auxiliary LFSR. Leveraging the sponge construction, this mechanism achieves the simultaneous absorption of physical entropy and squeezing of configuration bits.

Assuming the Strong PUF comprises n stages, the Auxiliary LFSR is configured with order $m = \log_2 n$. The complete configuration generation process requires a total of $n - 1$ clock cycles. The n-bit physical entropy E_{raw} is partitioned into two functional segments:

- Initial Seed: The first m bits, utilized for parallel seed loading.
- Injection Stream: The remaining $n - m$ bits, utilized for serial entropy injection.

For clarity, we elucidate the operating principle using a typical 64-bit implementation ($n = 64, m = 6$), as illustrated in Fig. 4. The specific workflow proceeds as follows:

- State Initialization ($t = 0$). The first 6 bits of the physical entropy vector (Initial Seed) are directly loaded into the registers ($x_0 \sim x_5$) of the Auxiliary LFSR, establishing the intrinsic initial state $X[0]$ of the circuit.
- Duplex Squeezing ($t = 1 \sim 58$). Governed by a primitive polynomial (e.g., $x^6 + x + 1$), the Auxiliary LFSR executes a 58-cycle serial injection. In each cycle t, the injection bit $u[t]$ is XORed with the feedback loop to yield the modulated feedback signal $\tilde{f}[t]$, which concurrently updates the state registers. This process ensures immediate entropy diffusion across the state space, while the leading state bit $x[t]_0$ is simultaneously extracted as the configuration output $g[t]$.
- Residual Squeezing ($t = 59 \sim 63$). Upon exhausting the injection stream, the input is constrained to Logic '0', effectively shifting the Auxiliary LFSR into a free-running mode. This guarantees that the entropy injected in the final moments continues to undergo deep mixing and diffusion, while the extraction logic remains invariant: the leading state bit $x[t]_0$ continues to be squeezed out to form the tail of the configuration sequence G.

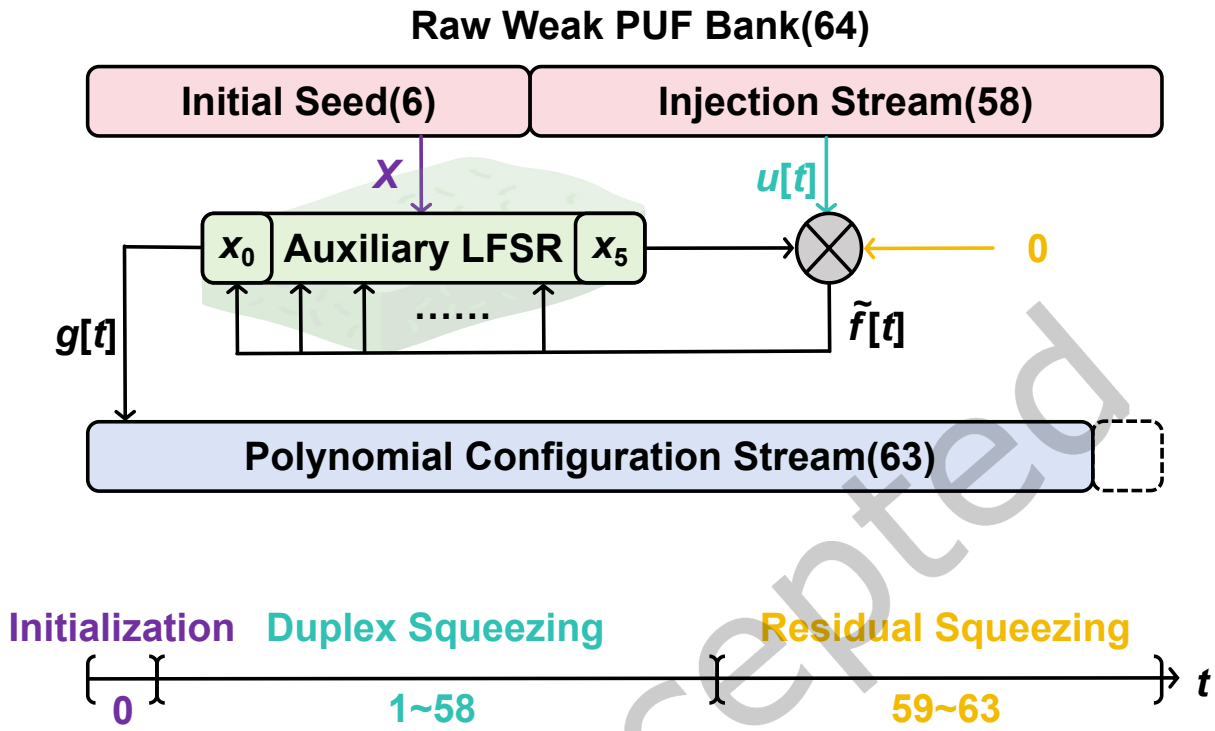

Fig. 4. Architecture and timing diagram of the Sponge-Based Configuration Mechanism (64-bit implementation).

Furthermore, the architecture is intrinsically scalable. For a 128-bit variant ($n = 128$), the Auxiliary LFSR expands to $m = 7$ stages. The mechanism generates the required 127 configuration bits by simply adjusting the timeline to 121 injection cycles and 6 residual squeezing cycles (121 + 6 = 127), maintaining the identical fundamental logic.

This design effectively obfuscates the direct dependency between the raw Weak PUF responses and the configuration parameters with minimal hardware cost. The primary hardware overhead stems from the Auxiliary LFSR, which exhibits logarithmic scaling ($O(\log n)$) with the PUF size. Consequently, the total additional area footprint is virtually negligible compared to the main PUF circuit. Admittedly, the serial configuration mechanism introduces an initialization latency of $n - 1$ clock cycles. However, this represents a strictly one-time setup cost. Once configured, the system performs continuous authentications for massive CRPs without reconfiguration, similar to conventional PUFs. When this marginal initialization time is amortized over thousands of subsequent authentication sessions, its impact on the average system throughput approaches zero and remains imperceptible in practical applications.

3.2 Obfuscation Process

This section details the core obfuscation mechanism. As visually summarized in Fig. 5, the entire procedure is temporally divided into two tightly coupled phases: the Response-Modulated State Evolution Phase (Fig. 5(1)) and the Hybrid Sequence Fusion (Fig. 5(2)).

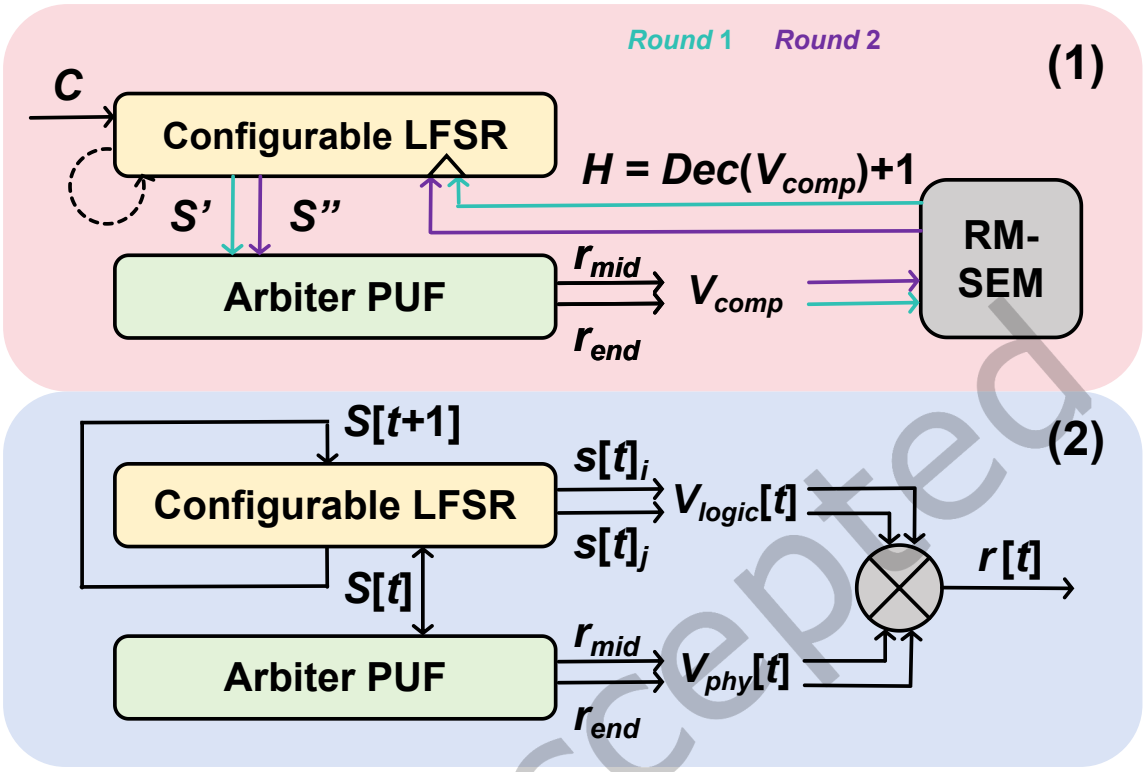

Fig. 5. Schematic overview of the proposed obfuscation mechanism.

3.2.1 Response-Modulated State Evolution Phase. Once the reconfigurable LFSR is initialized via the configuration stream driven, the system transitions into the state evolution phase. The process begins with the loading of the initial public challenge, denoted as C, into the reconfigurable LFSR. Immediately following this, a warm-up cycle is executed. During this cycle, the LFSR evolves purely based on its internal secret polynomial configuration without generating external output, transforming the initial state from C to a decorrelated internal state S'. Crucially, it is this evolved state S', rather than the public challenge C, that serves as the actual challenge applied to the APUF circuit, breaking the linear dependency between the challenge C and the subsequent responses.

To transform the spatial entropy of the APUF into temporal control logic, we define a core state-update rule utilized in the subsequent phases. Specifically, in each step, instead of relying on a single output bit, we extract a composite state vector V_{comp} by tapping into both the intermediate and final stages of the APUF MUX chain, defined as:

$$V_{comp} = \{r_{mid}, r_{end}\} \tag{7}$$

This vector is then mapped to a dynamic step-size parameter H by the Response-Modulated State Evolution Mechanism (RM-SEM). To prevent state stagnation and ensure continuous system evolution, we apply a simple affine transformation:

$$H = \text{Decimal}(V_{comp}) + 1 \tag{8}$$

This formulation maps the 2-bit entropy source into a non-zero shift domain $H \in \{1, 2, 3, 4\}$. As depicted by the control arrow in Fig. 5(1), the LFSR executes H consecutive clock cycles of state transitions based on this

parameter. This guarantees that the system undergoes a minimum of one non-linear permutation step for every challenge evaluation, thereby avoiding static stagnation points and minimizing unnecessary latency overhead.

Leveraging this rule, the system executes a dual-round strategy to achieve an optimal balance between prediction resilience and latency overhead:

- Round 1: The LFSR, holding the state S', stimulates the APUF to generate the first step-size H_1. The LFSR then shifts H_1 times, evolving its internal state from S' to an intermediate state S''. This iteration injects initial entropy into the system, initiating state divergence. However, a single round may still retain weak linear dependencies.
- Round 2: The evolved state S'' from the previous round serves as the new physical challenge for the APUF. This generates a second step-size H_2, driving the LFSR to evolve further. This maximizes non-linearity without incurring the excessive latency penalty typically associated with multi-round iterations.

This mechanism introduces non-deterministic state transitions at the physical layer: the state of the LFSR is strongly coupled to the historical trajectory of the PUF's internal responses via the feedback loop. By coupling the shift cycles with the previous response derived from the hardware's intrinsic process variations, we establish a strong temporal correlation.

3.2.2 Hybrid Sequence Fusion. Upon completion of the previous phase, the final resident state of the CLFSR is formally designated as the initial state vector $S[0]$. The system transitions into the final response generation phase, which is modeled as an iterative discrete-time system, as illustrated in Fig. 5(2). Let $S[t]$ denote the internal state vector of the LFSR at the t-th generation cycle, where $t = 0, 1, \ldots, L-1$.

At the beginning of each clock cycle, the current state vector $S[t]$ is not merely internal static data in a register but is redefined as the system's shared challenge source. It acts as a common heartbeat driving both worlds:

- Physical Extraction: $S[t]$ is applied directly as the physical challenge to the APUF. Leveraging intrinsic process variations, the system captures the physical response vector $V_{phy}[t]$ by sensing two specific nodes from the delay chain (consistent with Phase 1). This vector is explicitly composed of the responses from the intermediate and final stages:

$$V_{phy}[t] = [r_{mid}[t], r_{end}[t]]^T \tag{9}$$

- Logical Extraction: To ensure information balance, we sample two spatially separated bits from $S[t]$ to construct the logical state vector $V_{log}[t]$. As explicitly labeled in Fig. 5(2), this vector is defined as:

$$V_{log}[t] = [s[t]_i, s[t]_j]^T \tag{10}$$

  where indices i and j refer to disjoint positions (e.g., bits 16 and 48 in a 64-bit register) to reduce local correlation.

To deeply entangle physical delay characteristics with digital logic properties, the extracted vectors are immediately fed into a whitening mixer. The final response bit $r[t]$ is generated via a one-time multi-way XOR operation:

$$r[t] = V_{phy}[t]_0 \oplus V_{phy}[t]_1 \oplus V_{log}[t]_0 \oplus V_{log}[t]_1 \tag{11}$$

The raw output of an APUF may contain slight uniformity biases (i.e., 0/1 imbalance). As long as the logical vector $V_{log}[t]$ possesses good pseudo-random distribution properties, the XOR operation effectively masks any statistical defects of the hardware, thereby yielding a statistically uniform random sequence.

Concurrent with output generation, the LFSR computes the next state $S[t+1]$ based on $S[t]$ using its internal feedback polynomial logic. This creates the rolling update loop depicted on the left side of Fig. 5(2), where $S[t+1]$ serves as both the result of the previous evolution and the shared seed for the next generation.

By configuring the system to execute for L consecutive cycles (e.g., $L = 64$), this mechanism significantly expands the response space, greatly enhancing throughput. Crucially, this efficiency gain is underpinned by

robust security properties: the whitening mechanism maintains statistical independence between adjacent bits, and the generation of the t-th bit inherently relies on the full evolutionary trajectory of the previous $t-1$ cycles, thereby establishing a cumulative depth of confusion.

3.3 Algorithm and Numerical Walk-through

To facilitate a concrete understanding of the proposed architecture, we first formalize the complete execution procedure in Algorithm 1.

Algorithm 1: Proposed Sponge-Based Configuration and Obfuscation Process

Input : Public Challenge C, Physical Entropy E_{raw}
Output : Obfuscated Response Sequence $R_{seq} = \{r[0], \ldots, r[L-1]\}$

Partition E into Initial Seed X and Injection Stream U;
Initialize Aux LFSR state: $x \leftarrow X$;
for $i \leftarrow 1$ **to** $n-1$ **do**
  if $i \leq n-m$ **then**
    $b \leftarrow u[i]$;
  else
    $b \leftarrow 0$;
  $\tilde{f} \leftarrow \text{Feedback}(x) \oplus b$;
  $x \leftarrow \text{Shift}(x, \tilde{f})$;
  $g[i] \leftarrow x_0$;
  Shift $g[i]$ into Main LFSR Configuration Register;
Load C into Main LFSR: $S \leftarrow C$;
$S \leftarrow \text{LFSR\_Update}(S)$;
for $round \leftarrow 1$ **to** 2 **do**
  $V_{comp} \leftarrow \text{APUF\_Tap}(S, \{mid, end\})$;
  $H \leftarrow \text{Decimal}(V_{comp}) + 1$;
  for $step \leftarrow 1$ **to** H **do**
    $S \leftarrow \text{LFSR\_Update}(S)$;
Initialize empty sequence R_{seq};
for $t \leftarrow 0$ **to** $L-1$ **do**
  $V_{phy}[t] \leftarrow \text{APUF\_Tap}(S, \{mid, end\})$;
  $V_{log}[t] \leftarrow \{S_i, S_j\}$;
  $r[t] \leftarrow V_{phy}[t]_0 \oplus V_{phy}[t]_1 \oplus V_{log}[t]_0 \oplus V_{log}[t]_1$;
  Append $r[t]$ to R_{seq};
  $S \leftarrow \text{LFSR\_Update}(S)$;
return R_{seq};

Complementing the formal definition above, we present a simplified numerical walk-through to visualize the internal state transitions. For clarity and compactness, we define the system parameters and operational rules first, followed by a concise symbolic derivation.

- **System Scale:** $n = 4$ (Main LFSR), $m = 2$ (Aux LFSR).
- **Inputs:** Physical Entropy $E = 1101_2$ (Seed $X = 11$, Stream $U = 01$); Public Challenge $C = 1001_2$.
- **Aux LFSR Update Logic:**

$$x_0' \leftarrow x_1 \oplus u[t], \quad x_1' \leftarrow x_0 \oplus x_1 \oplus u[t]$$

- **APUF Logic:** $r_{end} = \bigoplus_{i=0}^{3} s_i$ (Parity); $r_{mid} = s_0 \oplus s_1$.

The state transitions ($X = [x_1, x_0]$) and generated configuration bits (G) are derived as follows:

- **Cycle 1 ($u = 0$):** $11_2 \xrightarrow{\text{Update}} 01_2$. Output $g_1 = 1$.
- **Cycle 2 ($u = 1$):** $01_2 \xrightarrow{\text{Update}} 01_2$. Output $g_2 = 1$.
- **Cycle 3 (Residual, $u = 0$):** $01_2 \xrightarrow{\text{Update}} 10_2$. Output $g_3 = 0$.

Result: Configuration sequence $G = \{1, 1, 0\}$ (Active Taps: g_1, g_2).

Using $C = 1001_2$ ($[s_3, s_2, s_4, s_0]$) and the derived update logic ($s_0' \leftarrow s_3$, $s_1' \leftarrow s_3 \oplus s_0'$, $s_2' \leftarrow s_3 \oplus s_1'$, $s_3' \leftarrow s_2$):

$$S_{init} = 1001_2 \xrightarrow{\text{Warm-up}} S' = 0101_2$$

The system executes two rounds of non-linear permutation. The step size is determined by $H = \text{Decimal}(V_{comp}) + 1$.

- **Round 1:** Challenge $S' = 0101_2$.
  - $V_{comp} = [1, 0]$ (derived from $r_{mid} = 1, r_{end} = 0$) $\implies H_1 = 3$.
  - $0101_2 \xrightarrow{1} 1010_2 \xrightarrow{2} 0011_2 \xrightarrow{3} 0110_2$.
- **Round 2:** Challenge $S'' = 0110_2$.
  - $V_{comp} = [1, 0]$ (derived from $r_{mid} = 1, r_{end} = 0$) $\implies H_2 = 3$.
  - $0110_2 \xrightarrow{1} 1100_2 \xrightarrow{2} 1111_2 \xrightarrow{3} 1001_2$.

Extracting logical bits at indices $i = 0, j = 2$ (Vector order $V_{log} = [s_2, s_0]$).

- **Cycle 0 ($t = 0$):** State 1001_2.
  - $V_{phy} = [1, 0]$, $V_{log} = [0, 1]$. Output $r[0] = 1 \oplus 0 \oplus 0 \oplus 1 = \mathbf{0}$.
  - Update State $\rightarrow 0101_2$.
- **Cycle 1 ($t = 1$):** State 0101_2.
  - $V_{phy} = [1, 0]$, $V_{log} = [1, 1]$. Output $r[1] = 1 \oplus 0 \oplus 1 \oplus 1 = \mathbf{1}$.
  - Update State $\rightarrow 1010_2$.

Final Sequence: $R_{seq} = \{0, 1\}$.

3.4 Reliability Enhancement Mechanism for Butterfly PUF

This work utilizes the Butterfly PUF architecture illustrated in Fig. 6. We explicitly define the propagation delay difference as $\Delta T = T_{top} - T_{bottom}$, where T_{top} and T_{bottom} denote the delays of the top and bottom paths, respectively. Although nominally symmetric, the path delay results from the accumulation of numerous microscopic stochastic variations. According to the Central Limit Theorem, ΔT follows a Gaussian distribution with a mean of zero and a standard deviation of σ [25]:

$$\Delta T \sim \mathcal{N}(0, \sigma^2) \tag{12}$$

As illustrated in Fig. 7, the horizontal axis represents the delay difference ΔT, while the vertical axis represents the probability density. When $\Delta T < 0$ (top path is faster), the race outcome tends toward Logic '0'; conversely, when $\Delta T > 0$, the outcome tends toward Logic '1'.

In practical modeling, the total effective delay difference implies a linear superposition of intrinsic process mismatch and environmental noise: $\Delta T_{total} = \Delta T_{process} + \Delta T_{noise}$. It is crucial to note that noise-induced jitter is bounded within a maximum envelope, denoted as $|\Delta T_{noise}^{max}|$. Based on the interplay between process variation and bounded noise (see Fig. 7), PUF cells are categorized as:

- Unreliable Cells ($|\Delta T_{process}| < |\Delta T_{noise}^{max}|$): The intrinsic mismatch is insufficient to withstand noise, allowing ΔT_{total} to cross zero and trigger bit flips.
- Reliable Cells ($|\Delta T_{process}| > |\Delta T_{noise}^{max}|$): The intrinsic margin dominates the worst-case noise, ensuring a stable output regardless of environmental fluctuations.

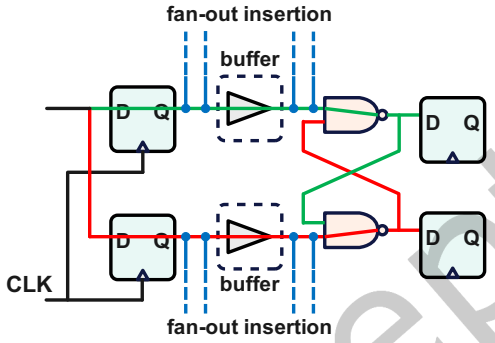

Fig. 6. Schematic of the proposed Butterfly PUF cell. The race condition determines the output bit.

To efficiently pre-characterize $\Delta T_{process}$ and identify robust candidates without prolonged statistical testing, we adopt the delay difference test proposed by Xu et al. [29]. This methodology subjects PUF cells to a stress test by introducing a configurable delay bias into the feedback paths. Instead of measuring absolute delays, it employs a strategy of deterministic emulation to quantify the stability margin of each cell against the noise envelope.

Functionally, we introduce a tunable delay bias $\delta_{load} \approx N \cdot \delta_{unit}$ to both chains. A cell is deemed reliable only if its output remains invariant under this bidirectional stress, mathematically guaranteeing that the intrinsic mismatch dominates the noise envelope ($|\Delta T_{process}| > |\Delta T_{noise}^{max}|$). Statistically, this mechanism establishes an exclusion zone of width $2N\delta_{unit}$ centered at the mean of the delay distribution. Consequently, the theoretical Qualification Rate, $R(N)$, is defined as the total probability mass falling outside this unstable region, formulated as:

$$R(N) = 1 - \int_{-N\cdot\delta_{unit}}^{+N\cdot\delta_{unit}} \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}}\, dx \tag{13}$$

As implied by Eq. 13, a critical trade-off exists: increasing N initially sharpens reliability by eliminating the highly unstable cells near the peak density ($\mu = 0$). However, excessive N (beyond the 3σ noise bound) yields diminishing returns in stability while exponentially reducing yield. Empirical results from Xu et al. [29] validate that $N = 4$ offers the optimal balance between reliability saturation (100%) and selection efficiency.

Beyond yield trade-offs, ensuring statistical unbiasedness is critical. Since the Gaussian PDF $f(x)$ is an even function and the screening applies a symmetric exclusion threshold $\pm N\delta_{unit}$, the remaining tail probabilities are strictly equal:

$$P(0) = \int_{-\infty}^{-N\delta_{unit}} f(x)\, dx = \int_{+N\delta_{unit}}^{+\infty} f(x)\, dx = P(1) \tag{14}$$

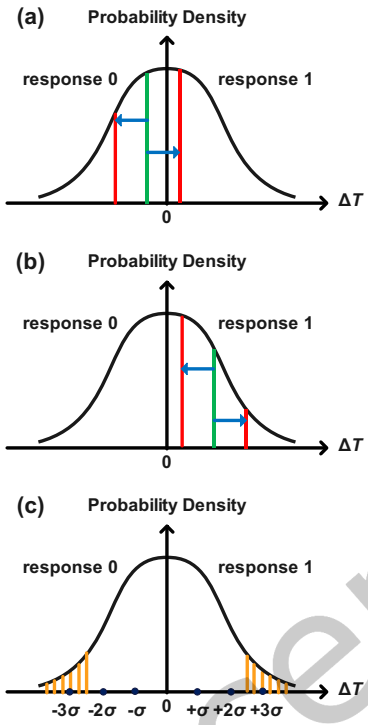

Fig. 7. Probability density distribution of the delay difference ΔT. (a) Probability density under process variations, highlighting the unreliable central region; (b) Reliable regions where intrinsic mismatch exceeds bounded noise; (c) Symmetric integration regions of the tails

This symmetry guarantees a perfect 50% balance between Logic 0 and 1. Furthermore, for a zero-mean Gaussian distribution, the sign (output bit) and magnitude (reliability) of ΔT are statistically independent. Thus, the mechanism eliminates instability while fully preserving entropy, as confirmed by NIST tests in Xu et al. [29].

It is imperative to distinguish between the storage of reliability information and the storage of secret responses. The proposed screening mechanism generates a configuration mask (helper data) that strictly records the coordinate indices of the reliable PUF cells within the FPGA fabric, rather than their binary evaluation results. As analyzed in the previous section, the reliability (magnitude of the intrinsic delay variation) contains no information about the response (polarity of the variation). Therefore, storing this selection mask exposes only the location of robust resources, which is statistically independent of the secret key material. Even if an attacker obtains this location information, they only learn that a specific PUF cell is stable, while the actual logical output ('0' or '1') remains entirely unknown. This approach adheres to the fundamental PUF premise: the secret is not statically stored but is re-generated on-the-fly by measuring the specific FPGA instance's physical characteristics upon power-up.

4 Security Analysis

In this section we quantitatively evaluate the security resilience against modeling attacks from three complementary perspectives: information-theoretic uncertainty, computational complexity, and gradient-based learning resistance.

4.1 Information Uncertainty

The security of the proposed architecture follows Kerckhoffs's principle. The system's security relies entirely on the secrecy of the Key residing within the chip. Here, the raw physical fingerprint E_{raw} generated by the Weak PUF array acts as this unique private key (K). Consequently, the core system configurations—the feedback polynomial G and the state transition matrix $\mathbf{M}$—are defined by microscopic process variations and remain as unknown Hidden Variables to the attacker.

For traditional Strong PUFs (e.g., APUF), the response R is a deterministic function of the challenge C. Given the deterministic delay model, knowing C provides full information to determine R, which implies high mutual information, i.e., $I(R;C)$ can be approximated as 1 bit. However, in our design, the mapping is dependent on the hidden key K. From an attacker's perspective, the system output R is an encryption function:

$$R = \mathcal{F}(C, \mathcal{T}_{history} \mid K = E_{raw}) \tag{15}$$

Without physical extraction of E_{raw}, knowledge of the public challenge C provides no advantage in predicting R. The mutual information drops to near zero:

$$I(R; C \mid \text{unknown } E_{raw}) \approx 0 \tag{16}$$

This indicates that without the key, the observed CRP data is statistically equivalent to independent noise. This Keyed-PRF characteristic theoretically cuts off the feature extraction path for machine learning models.

4.2 Complexity Explosion via Structural Interdependency

Standalone components inherently suffer from mathematical vulnerabilities: LFSRs allow for structural recovery via the Berlekamp-Massey (B-M) algorithm–a classic method that can efficiently deduce the internal feedback structure from the output sequence–due to their linear predictability, while APUFs are susceptible to linear modeling attacks. However, by tightly coupling the physical and logical layers, the proposed design establishes a structural interdependency that overcomes these individual weaknesses.

- For LFSR, the internal state sequence is masked to preclude direct mathematical analysis, with $S[t]$ serving as a dynamic challenge driving the APUF. By applying a non-linear step function (sgn) to the accumulated physical delays and XORing the result with the logical state, the feedback loop incorporates strict non-linearity. This transforms the attack surface from a linear system into a computationally hard Non-Linear Feedback Shift Register (NLFSR) state recovery problem, typically requiring an exhaustive search.
- Conversely, the logic layer insulates the APUF from modeling attacks. The LFSR's large state space acts as a rapidly changing, unpredictable excitation source. Crucially, both the feedback polynomial and the evolution step size are dynamically determined by unknown physical process variations. Consequently, the effective challenge sequence applied to the APUF appears statistically random to an attacker. This prevents the collection of stable CRPs necessary for gradient-based model fitting.

Consequently, the computational resources C_{attack} required to infer the hidden state scale exponentially with the LFSR width n:

$$C_{attack} \propto O(2^n) \tag{17}$$

Given that n is typically configured to 64 or 128 in our design, this establishes a formidable complexity barrier rooted in the massive size of the logical state space.

4.3 Gradient Blocking via Hidden Markov Chains

Most supervised learning attacks (e.g., LR, DNN) rely on the core assumption that training data are Independent and Identically Distributed (IID). However, our design introduces full-history dependency, creating a deep

temporal system. The system operates as a dynamical system governed by state-space equations, where the current response $r[t]$ strictly relies on the previous internal state $S[t-1]$. This creates a recursive dependency chain extending back to the initialization. Since the internal state vector $S[t]$ is doubly masked by the physical layer (APUF delay) and the logical layer (XOR whitening), it constitutes a Hidden State.

From a modeling perspective, this generates a Hidden Markov Model (HMM). For an attacker, determining the mapping function shifts from a simple approximation problem to the extremely challenging task of inferring hidden states in a high-dimensional space. Since the state transition logic is modulated by unknown physical process variations, the transition probabilities are opaque to the attacker. Without recovering the precise sequence of historical states, standard gradient descent algorithms fail to compute the correct gradient direction, leading to non-convergence. This mechanism effectively exploits the lack of contextual information in conventional ML models to block modeling attacks.

5 PUF Performance Simulation

To preliminarily evaluate the statistical properties and theoretical robustness of the proposed PUF, we developed a simulation framework using Python 3.10. The delay parameters of the APUF components are modeled as independent and identically distributed Gaussian variables obeying $N(0,1)$. To simulate realistic operational conditions, we introduced additive Gaussian noise $N(0,\sigma^2_{noise})$, where the noise intensity is defined proportional to the process variation ($\sigma_{noise} = \eta \cdot \sigma_{process}$). We selected $\eta \in \{0.1, 0.2, 0.3\}$ to represent scenarios ranging from ranging from severe interference to worst-case extreme disturbances. Furthermore, the auxiliary weak PUF array is assumed to be stable and immune to noise interference. Given the structurally invariant generation logic, the output stream approximates a stationary stochastic process, rendering the first bit $r[0]$ statistically representative of the entire sequence, while simultaneously serving as the security lower bound due to minimal historical entropy.

5.1 Correlation Analysis Between Intermediate and Final Responses

To validate the effectiveness of the proposed response-modulated mechanism, it is essential to confirm that the intermediate response (r_{mid}) and the final response (r_{end}) are statistically independent. Although r_{end} physically shares a partial delay path with r_{mid}, we hypothesize that the accumulated process variation in the subsequent stages acts as a dominant entropy source, effectively decorrelating the two outputs.

We quantified this independence by conducting a Monte Carlo simulation with 1000 independent APUF instances. For each instance, we tested the PUF using 20,000 random challenges and computed the Fractional Hamming Distance (FHD) between the r_{mid} and r_{end} sequences.

As illustrated in Fig. 8, the distribution of FHD values approximates a Gaussian distribution, tightly clustered around the ideal value of 50% ($\mu \approx 50.00\%, \sigma \approx 0.005$). This empirical result confirms that the structural coupling does not compromise statistical independence, as the randomness introduced by the second half of the circuit effectively masks the bias from the first half.

5.2 Uniformity and Uniqueness Analysis

In our simulation setup, we instantiated 100 independent PUF instances and applied 20,000 random challenges to each. To verify the system's robustness under varying environmental conditions, we evaluated the performance of both 64-stage and 128-stage architectures under three distinct noise ratios.

The experimental results are presented in Fig. 9, utilizing split violin plots to visualize the probability density of HW and Inter-Die HD. As illustrated, the proposed design exhibits excellent statistical characteristics. For both 64-bit and 128-bit configurations, the distributions of HW and HD are tightly clustered around the ideal 50% baseline. Across all tested noise scenarios, the mean values strictly fluctuate within the narrow range of

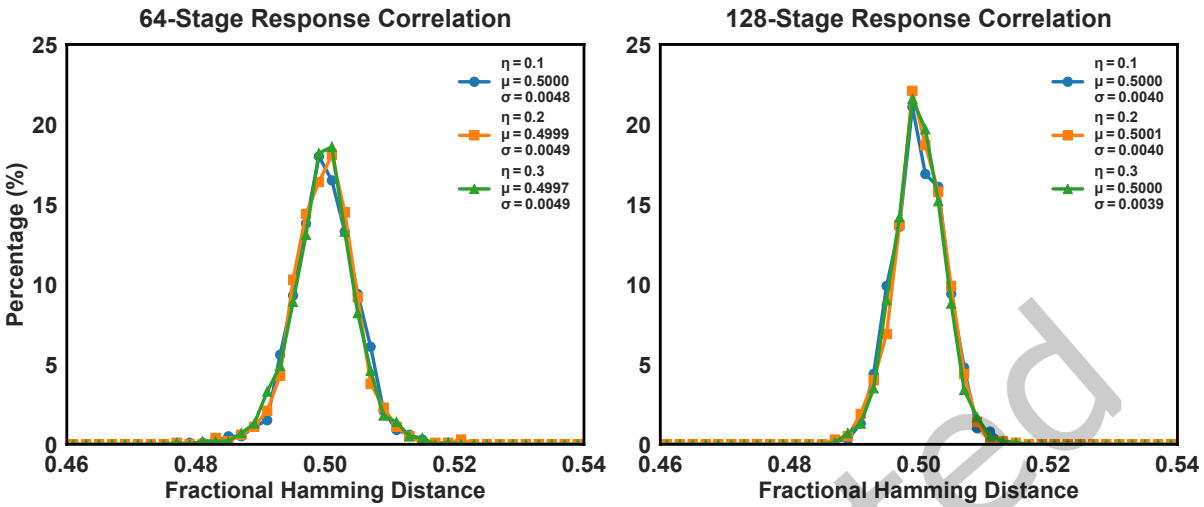

Fig. 8. Statistical independence analysis of APUF.

49.92% to 50.03%, while the standard deviations are consistently suppressed below 0.46%. Even as the noise ratio η increases to 0.3, the distributions show no significant degradation or bias shift.

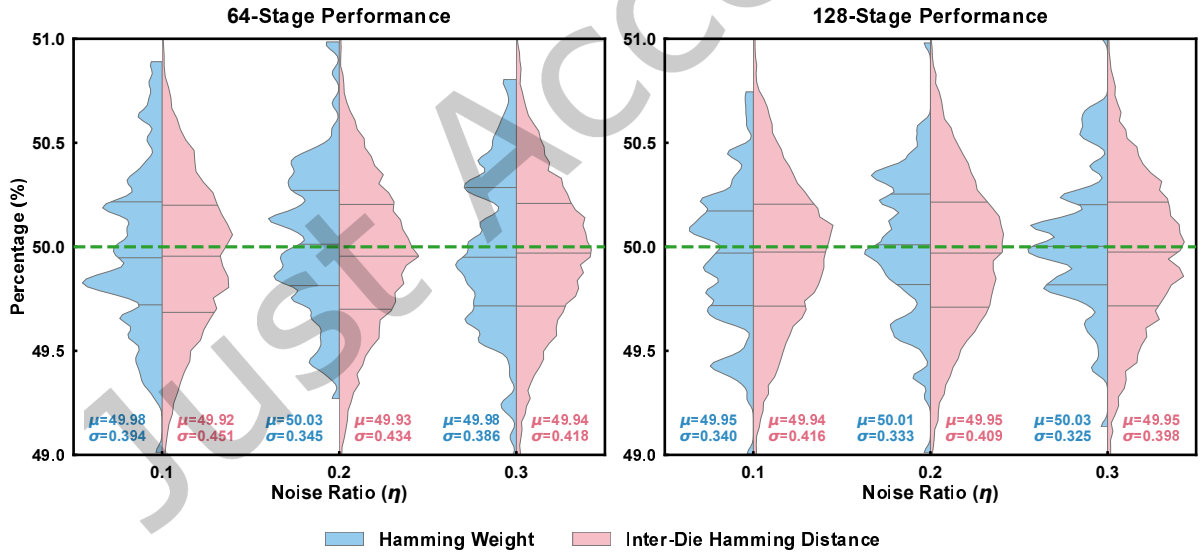

Fig. 9. Statistical performance of Uniformity (Hamming Weight) and Uniqueness (Inter-Die Hamming Distance).

The near-ideal uniformity is guaranteed by the XOR whitening operation in the hybrid sequence fusion phase, which effectively masks physical biases.Meanwhile, the superior uniqueness observed is primarily attributed to the fact that each APUF is coupled with a device-specific LFSR. Specifically, the CLFSR in each device is

governed by a unique set of feedback taps derived from the intrinsic Weak PUF. This unique configuration ensures that, even under identical external challenges, the internal state evolution trajectories of different chips diverge completely. Consequently, this mechanism generates highly uncorrelated response sequences, thereby guaranteeing distinct inter-die fingerprints.

5.3 Reliability Optimization via Pre-screening Strategy

In the proposed design, the evolution step size H of the LFSR is dynamically determined by the instantaneous physical responses (r_{mid}, r_{end}) of the APUF. Under environmental noise, even a single bit flip in these feedback responses triggers a catastrophic error propagation: the incorrect step size directs the LFSR to a divergent state trajectory, severing the cryptographic binding between the challenge and response. This logic propagation error, characterized by an avalanche effect, causes the Intra-HD of the output to degrade toward the maximum entropy value of 0.5, rendering the system unsuitable for reliable authentication.

To address this challenge, we propose a pre-screening strategy based on Bit Error Rate (BER) tolerance. Specifically, we designed a parameter scanning experiment where a challenge is deemed "qualified" only if its maximum Intra-HD across $N = 10$ repeated measurements remains below a preset threshold T_{HD}. To construct a robust dataset, we employed a rejection sampling mechanism for each instance, iterating the generation process until 2,000 qualified challenges were collected. In this process, we utilized a response sequence of length $L = 100$ as the observation metric; compared to single-bit measurements, this metric offers higher statistical confidence, enabling more precise identification and removal of unstable challenges. We simulated 100 PUF instances and conducted a full-range sweep of T_{HD} from 0.95 to 0.1 under various noise environments. Table 1 presents the comprehensive statistical results of the experiment. Specifically, it details the evolution of the qualification rate (defined as the proportion of challenges retained from the randomly generated pool) and the post-screening Intra-HD characteristics.

Based on the parameter sweep results presented in Table 1, we identify $T_{HD} = 0.40$ as the optimal screening threshold for the system design. This selection derives from a quantitative trade-off analysis between error suppression efficacy and challenge yield:

- The convergence trend of Intra-HD reveals that the most significant error reduction occurs as the threshold tightens towards 0.40 (e.g., Intra-HD decreases from 0.1653 to 0.1206 for the 64-bit system at $\eta = 0.2$). However, as the threshold falls below 0.40 (e.g., to 0.35 or 0.30), the improvement in Intra-HD becomes negligible, exhibiting characteristics of diminishing marginal utility. This implies that at $T_{HD} = 0.40$, the strategy has effectively isolated logic propagation errors, with the residual error dominated by the intrinsic physical noise floor. Further tightening of the threshold results in ineffective screening of inherent physical noise without substantial stability gains.
- From the perspective of challenge yield, 0.40 serves as the critical boundary for maintaining the efficiency of dataset construction. At $T_{HD} \geq 0.40$, the system sustains a viable yield of approximately 63% even under $\eta = 0.2$. Conversely, crossing this boundary to $T_{HD} = 0.20$ triggers a precipitous drop in yield (dropping to approx. 10%–11% at $\eta = 0.3$).

Consequently, $T_{HD} = 0.40$ maximizes the elimination of logic avalanche errors while avoiding the excessive computational overhead associated with low qualification rates.

To further verify the universality and effectiveness of the selected threshold $T_{HD} = 0.40$, we conducted a reliability evaluation across 100 independent PUF instances. We compared the reliability distribution of the screened dataset against a randomly generated unscreened dataset (both containing 2,000 CRPs, measured 10 times). The statistical results are presented in Fig. 10.

The evaluation reveals a significant optimization in the population reliability profile driven by the pre-screening strategy: The entire reliability distribution shifts noticeably towards the ideal value of 1.0, which confirms that

Table 1. Comprehensive Statistics of Qualification Rate and Intra-HD

(a) 64-Stage PUF Architecture

Threshold	Noise $\eta = 0.1$		Noise $\eta = 0.2$		Noise $\eta = 0.3$	
(T_{HD})	Qual. Rate (%)	Intra-HD (After)	Qual. Rate (%)	Intra-HD (After)	Qual. Rate (%)	Intra-HD (After)
0.95	100.00 ± 0.00	0.0859 ± 0.0607	100.00 ± 0.00	0.1653 ± 0.0755	100.00 ± 0.00	0.2210 ± 0.0702
0.85	100.00 ± 0.00	0.0859 ± 0.0607	100.00 ± 0.00	0.1653 ± 0.0755	100.00 ± 0.00	0.2210 ± 0.0702
0.75	99.99 ± 0.01	0.0859 ± 0.0607	99.98 ± 0.01	0.1654 ± 0.0757	99.98 ± 0.01	0.2210 ± 0.0702
0.65	99.12 ± 0.11	0.0848 ± 0.0587	98.30 ± 0.18	0.1628 ± 0.0735	97.50 ± 0.22	0.2190 ± 0.0692
0.60	97.01 ± 0.29	0.0811 ± 0.0539	94.21 ± 0.51	0.1572 ± 0.0692	91.54 ± 0.65	0.2131 ± 0.0657
0.55	90.38 ± 0.88	0.0730 ± 0.0407	81.56 ± 1.52	0.1392 ± 0.0500	73.51 ± 1.96	0.1920 ± 0.0515
0.50	84.04 ± 1.40	0.0654 ± 0.0253	69.89 ± 2.39	0.1255 ± 0.0324	57.51 ± 3.00	0.1761 ± 0.0356
0.45	81.71 ± 1.58	0.0638 ± 0.0221	65.76 ± 2.66	0.1224 ± 0.0288	52.03 ± 3.24	0.1722 ± 0.0322
0.40	80.55 ± 1.66	0.0629 ± 0.0209	63.71 ± 2.80	0.1206 ± 0.0274	49.35 ± 3.39	0.1709 ± 0.0312
0.35	79.95 ± 1.70	0.0626 ± 0.0206	62.65 ± 2.87	0.1200 ± 0.0273	47.39 ± 3.87	0.1698 ± 0.0305
0.30	79.86 ± 1.70	0.0626 ± 0.0206	62.35 ± 2.99	0.1197 ± 0.0271	44.32 ± 5.45	0.1667 ± 0.0280
0.20	79.58 ± 1.87	0.0623 ± 0.0203	47.38 ± 8.72	0.1104 ± 0.0220	11.11 ± 7.06	0.1336 ± 0.0191
0.10	41.60 ± 9.48	0.0496 ± 0.0133	1.37 ± 1.57	0.0647 ± 0.0108	0.05 ± 0.01	0.0699 ± 0.0096

(b) 128-Stage PUF Architecture

Threshold	Noise $\eta = 0.1$		Noise $\eta = 0.2$		Noise $\eta = 0.3$	
(T_{HD})	Qual. Rate (%)	Intra-HD (After)	Qual. Rate (%)	Intra-HD (After)	Qual. Rate (%)	Intra-HD (After)
0.95	100.00 ± 0.00	0.0885 ± 0.0643	100.00 ± 0.00	0.1611 ± 0.0698	100.00 ± 0.00	0.2221 ± 0.0699
0.85	100.00 ± 0.00	0.0885 ± 0.0643	100.00 ± 0.00	0.1611 ± 0.0698	100.00 ± 0.00	0.2221 ± 0.0699
0.75	99.99 ± 0.01	0.0885 ± 0.0643	99.98 ± 0.01	0.1611 ± 0.0698	99.98 ± 0.01	0.2221 ± 0.0699
0.65	99.12 ± 0.09	0.0876 ± 0.0630	98.28 ± 0.15	0.1594 ± 0.0681	97.49 ± 0.17	0.2192 ± 0.0679
0.60	96.98 ± 0.21	0.0841 ± 0.0573	94.14 ± 0.36	0.1532 ± 0.0622	91.46 ± 0.48	0.2144 ± 0.0641
0.55	90.32 ± 0.54	0.0733 ± 0.0383	81.41 ± 0.95	0.1388 ± 0.0464	73.27 ± 1.28	0.1942 ± 0.0487
0.50	83.92 ± 0.86	0.0664 ± 0.0232	69.65 ± 1.47	0.1258 ± 0.0303	57.13 ± 1.86	0.1798 ± 0.0345
0.45	81.55 ± 0.98	0.0652 ± 0.0212	65.46 ± 1.64	0.1230 ± 0.0274	51.67 ± 2.02	0.1747 ± 0.0310
0.40	80.38 ± 1.03	0.0644 ± 0.0204	63.37 ± 1.72	0.1218 ± 0.0264	48.98 ± 2.11	0.1727 ± 0.0301
0.35	79.77 ± 1.06	0.0643 ± 0.0204	62.29 ± 1.76	0.1212 ± 0.0260	47.05 ± 2.44	0.1710 ± 0.0292
0.30	79.68 ± 1.06	0.0642 ± 0.0203	62.01 ± 1.83	0.1209 ± 0.0256	43.99 ± 3.63	0.1679 ± 0.0275
0.20	79.42 ± 1.17	0.0642 ± 0.0203	46.67 ± 6.22	0.1126 ± 0.0209	9.82 ± 4.64	0.1352 ± 0.0179
0.10	40.51 ± 6.74	0.0502 ± 0.0134	1.00 ± 0.74	0.0662 ± 0.0110	0.05 ± 0.01	0.0727 ± 0.0092

the threshold effectively filters out pathological challenges that induce instability across the chip population. The distribution post-screening appears significantly sharper, characterized by a substantial reduction in standard deviation, which indicates that not only has the overall performance improved, but the variation between different PUF instances has also been minimized. These results strongly demonstrate that $T_{HD} = 0.40$ is a robust engineering parameter.

5.4 Provable Learnability Assessment via Boolean Analysis

Following the theoretical framework of PUFmeter established by Ganji et al. [6], we conducted a rigorous property evaluation.

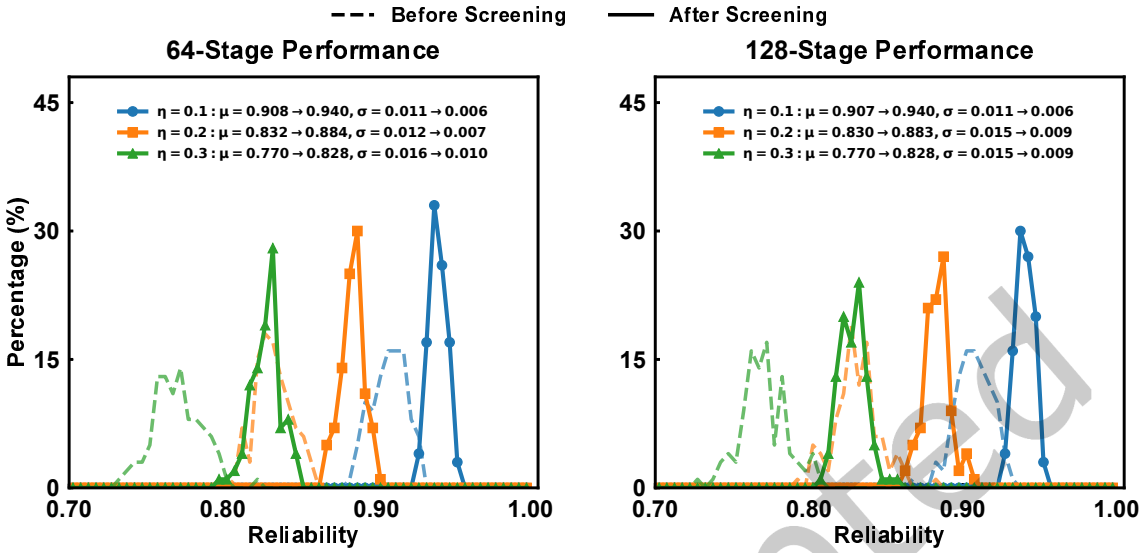

Fig. 10. Reliability Distribution Across 100 PUF Instances ($T_{HD} = 0.40$).

5.4.1 Average Sensitivity. The primary metric, Average Sensitivity ($I(f)$), quantifies the cumulative influence of all n input variables on the output, defined as the sum of the influence of individual bits:

$$I(f) = \sum_{i=1}^{n} Inf_i(f) = \sum_{i=1}^{n} \Pr_c[f(c) \neq f(c^{\oplus i})] \tag{18}$$

where $c^{\oplus i}$ denotes the challenge vector with the i-th bit flipped. From a cryptanalytic perspective, this metric serves as a rigorous security bound. If $I(f)$ is bounded by $O(\log n)$, the function theoretically degenerates into a k-Junta function (where the output depends on only a small subset of inputs), rendering it vulnerable to feature selection attacks. Therefore, a secure Strong PUF must satisfy the condition $I(f) \gg O(\log n)$, with the ideal value approaching $n/2$ (indicating satisfaction of the Strict Avalanche Criterion).

To verify this property, we evaluated 100 randomly instantiated PUF instances under three distinct noise regimes ($\eta \in \{0.1, 0.2, 0.3\}$). To rigorously isolate the structural Boolean properties from transient physical noise, we implemented a majority voting mechanism ($N_{rep} = 101$) to determine the stable response for every query. The sensitivity $I(f)$ was computed via Monte Carlo simulation using a baseline set of 5000 random challenges per instance. Specifically, we employed a bit-flipping strategy where we systematically iterated through and inverted each of the n input bits (for $n = 64$ and 128), quantifying the cumulative influence of each bit on the stabilized output.

The distribution of the measured Average Sensitivity for both 64-stage and 128-stage architectures is visualized in Fig. 11. For the 64-stage architecture, the mean sensitivity (μ) consistently clusters around 30.9; similarly, for the 128-stage configuration, it remains stable around 61.2. Although these values are slightly below the theoretical maximums ($n/2$, i.e., 32 and 64), they significantly exceed the learnability bounds for k-Junta functions ($O(\log 64) \approx 6$ and $O(\log 128) \approx 7$). Furthermore, the standard deviations (σ) are consistently suppressed at low levels (typically $\sigma < 0.71$ for 64-stage and $\sigma < 1.37$ for 128-stage) across all scenarios. Collectively, the high sensitivity levels and low variance confirm that the output depends on the global interaction of the challenge bits rather than a small subset, effectively ruling out structural weaknesses exploitable by junta-learning attacks.

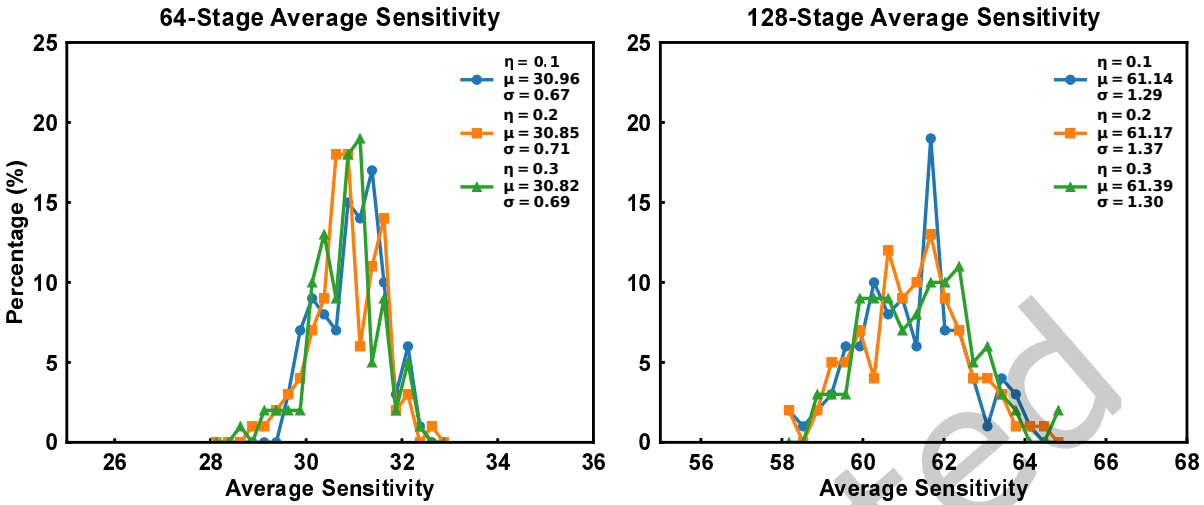

Fig. 11. Distribution of Average Sensitivity for 64-stage and 128-stage architectures.

5.4.2 Noise Sensitivity. Complementary to Average Sensitivity, we further evaluate the Noise Sensitivity ($NS_\epsilon(f)$), which measures the stability of the Boolean function under varying input perturbations. Formally, let c' be a noisy version of the challenge c, formed by flipping each bit of c independently with a probability $\epsilon \in (0, 1/2)$. The noise sensitivity is defined as the probability that the function output changes:

$$NS_\epsilon(f) = \Pr_{c,c'}[f(c) \neq f(c')] \tag{19}$$

From the perspective of Fourier analysis, if a Boolean function exhibits low noise sensitivity (specifically, bounded by $\sqrt{\epsilon}$), its Fourier spectrum is theoretically concentrated on low-degree coefficients, thereby rendering it susceptible to efficient approximation in polynomial time. Consequently, a robust Strong PUF must satisfy the condition $NS_\epsilon(f) \gg \sqrt{\epsilon}$. Ideally, the sensitivity should approach 0.5.

We evaluated the Noise Sensitivity using 100 randomly instantiated PUF instances. For each instance, the metric was computed using 5,000 random challenges. To emulate varying degrees of input perturbation, we calibrated the bit-flip probabilities ϵ based on the measured hardware performance. Crucially, this parameter ϵ corresponds to the effective Bit Error Rate (BER), which is mathematically equivalent to $1 - Reliability$. For each challenge c, we generated a perturbed version c' by independently flipping each bit with probability ϵ. We then compared the responses of the stabilized Boolean function to quantify the probability of output divergence. We then compared the responses of the stabilized Boolean function (i.e., $f(c)$ vs. $f(c')$) to quantify the probability of output divergence.

The distribution of measured Noise Sensitivity (NS_ϵ) is visualized in Fig. 12. Correlating empirical data with theoretical bounds reveals robust security characteristics:

- At the baseline noise level ($\eta = 0.1$), the theoretical bound $\sqrt{\epsilon}$ is approximately 0.2449, whereas the measured sensitivity reaches ≈ 0.48 for both architectures. This nearly 2× margin provides rigorous evidence that the design strictly violates LMN prerequisites under standard noise conditions.

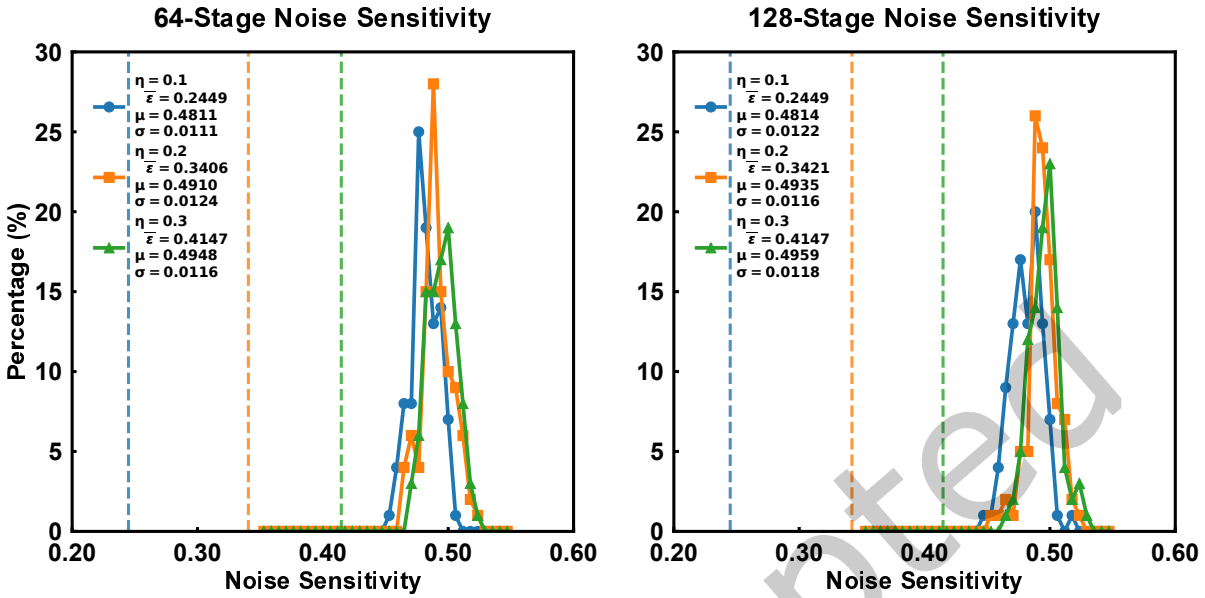

Fig. 12. Distribution of Noise Sensitivity (NS_ϵ) for 64/128-stage architectures. Dashed lines indicate theoretical learnability bounds ($\sqrt{\epsilon}$).

- As noise intensity increases, the gap between the measured sensitivity and the theoretical bound naturally narrows. This is a mathematical necessity rather than a design flaw: since the measured NS is physically capped at 0.5 (which our design approaches) while the theoretical bound $\sqrt{\epsilon}$ grows monotonically, convergence is inevitable as the bound approaches the physical ceiling.
- Since $\eta = 0.1$ represents a conservative high-noise scenario, lower noise levels ($\epsilon \to 0$) would only further expand the security margin. The theoretical bound decays parabolically while our sensitivity exhibits only negligible deviation from the ideal 0.5. Thus, these results effectively represent a lower bound on security, demonstrating absolute robustness in typical high-quality silicon environments.

6 FPGA Verification

To validate the feasibility and performance of the proposed architecture in silicon, we implemented the prototype on a Xilinx Artix-7 FPGA (XC7A100T) development board. The design flow, including logic synthesis and place-and-route, was executed using the Xilinx ISE 14.7 design suite. The APUF delay chains were constructed using MUX primitives within Slices, while the arbiter was implemented as a D Flip-Flop. A critical aspect of the implementation concerns the physical layout of the Weak PUF. We adopted a strict hard-macro placement strategy inspired by [29]. Specifically, manual layout constraints were applied to the PUF cells to guarantee the physical symmetry of the differential delay paths within the FPGA fabric. A MicroBlaze soft-core processor was implemented to facilitate data acquisition, serving exclusively as a UART bridge for exchanging CRPs with the host. Consistent with the Python simulation setup described in Section 5, the hardware evaluation focuses exclusively on the first response bit, r_0.

6.1 Uniformity and Uniqueness

To evaluate statistical performance, response sequences were acquired from 20 FPGA instances, applying a dataset of 2,000 random challenges to each. First, Uniformity was analyzed to assess the balance between logic '0's and '1's. As illustrated in Fig. 13, the HW for both 64-stage and 128-stage configurations cluster tightly around the ideal 50% baseline, achieving average uniformity scores of 50.18% and 49.90%, respectively. This indicates that the architecture effectively suppresses systematic bias. Second, Uniqueness was evaluated by calculating the Inter-HD to quantify device distinguishability. As depicted in Fig. 14, the HD distribution follows a near-perfect Gaussian curve centered very close to the ideal 50%. Specifically, the mean Uniqueness values were calculated as 49.91% for the 64-stage and 49.82% for the 128-stage design, confirming statistical independence across different chips. These silicon results strongly corroborate the simulation findings in Section 5.2.

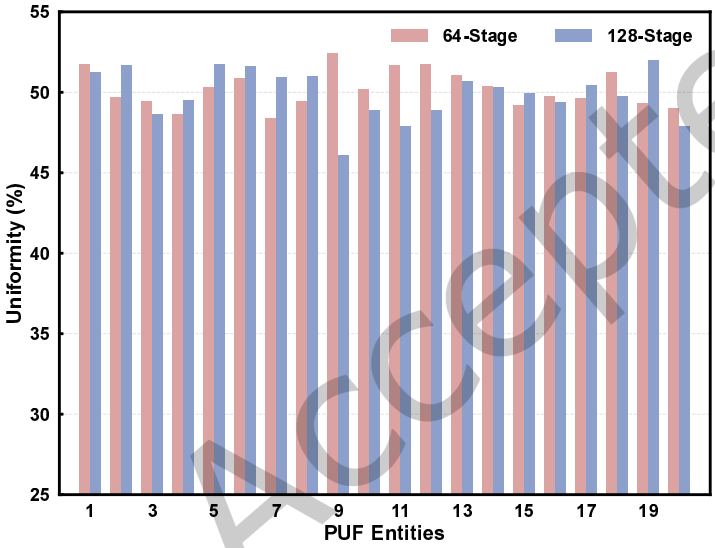

Fig. 13. Measured Uniformity across 20 PUF entities for 64-stage and 128-stage configurations.

6.2 Randomness

To rigorously evaluate the statistical quality of the generated responses, we utilized a 64-stage PUF instance to generate a contiguous stream of 1×10^6 bits (1M bits). The evaluation focuses on two critical dimensions: statistical randomness verified by the NIST SP 800-22 test suite, and correlation properties analyzed via the Autocorrelation Function (ACF).

6.2.1 NIST SP 800-22 Test Results. The generated bitstream was subjected to the standard NIST test suite consisting of 15 statistical tests. The evaluation criterion relies on the P-value; a sequence is considered to possess randomness statistically indistinguishable from a true random source if the computed P-value exceeds the significance level $\alpha = 0.01$. Table 2 summarizes the test results. As indicated, the proposed PUF successfully passes all 15 sub-tests. The P-values for all metrics, ranging from the *Frequency* test to the complex *Linear Complexity* test, are significantly higher than the threshold of 0.01. This confirms that the proposed architecture effectively suppresses the process variation biases and produces high-entropy responses suitable for cryptographic applications.

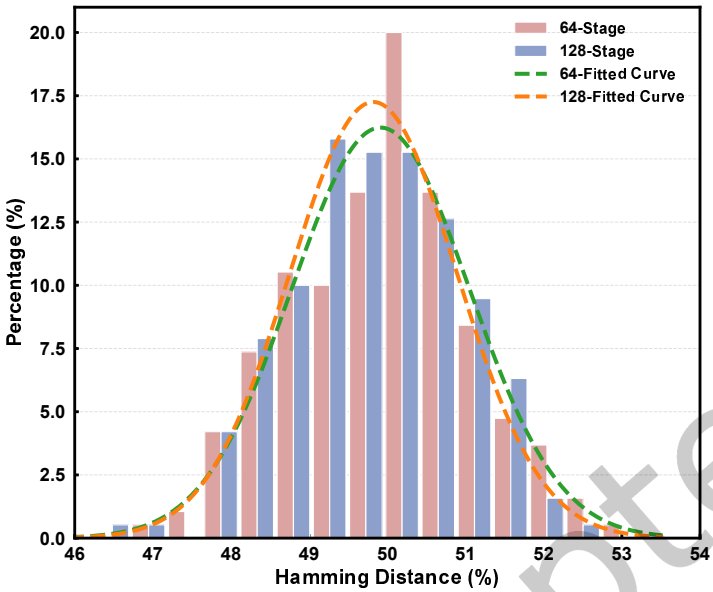

Fig. 14. Distribution of Inter-chip Hamming Distance (Uniqueness) with fitted Gaussian curves.

Table 2. NIST SP 800-22 Test Suite Results (Data Length: 1M bits)

Test Name	Result	P-value
Frequency	**Pass**	0.055620
Block Frequency	**Pass**	0.837418
Cumulative Sums	**Pass**	0.065230
Runs	**Pass**	0.966230
Longest Run	**Pass**	0.058327
Rank	**Pass**	0.178955
FFT	**Pass**	0.941477
Non Overlapping Template	**Pass**	0.505977
Overlapping Template	**Pass**	0.322578
Universal	**Pass**	0.038892
Approximate Entropy	**Pass**	0.537362
Random Excursions	**Pass**	0.455705
Random Excursions Variant	**Pass**	0.726591
Serial	**Pass**	0.908976
Linear Complexity	**Pass**	0.602566

6.2.2 Autocorrelation Analysis. To verify the temporal independence of the PUF responses, the ACF was evaluated. For an ideal random source, the autocorrelation coefficients at any non-zero lag ($k \neq 0$) should be statistically close to zero, exhibiting white noise characteristics.

As shown in Fig. 15, excluding the theoretical unit peak at zero lag, the correlation coefficients at all other lags fluctuate stochastically around zero and are bounded within the 95% confidence intervals (indicated by green solid lines). No significant periodic patterns were observed, confirming the statistical independence of the generated bitstrea and exhibiting no distinct linear correlation or memory effects even under large-scale data evaluation.

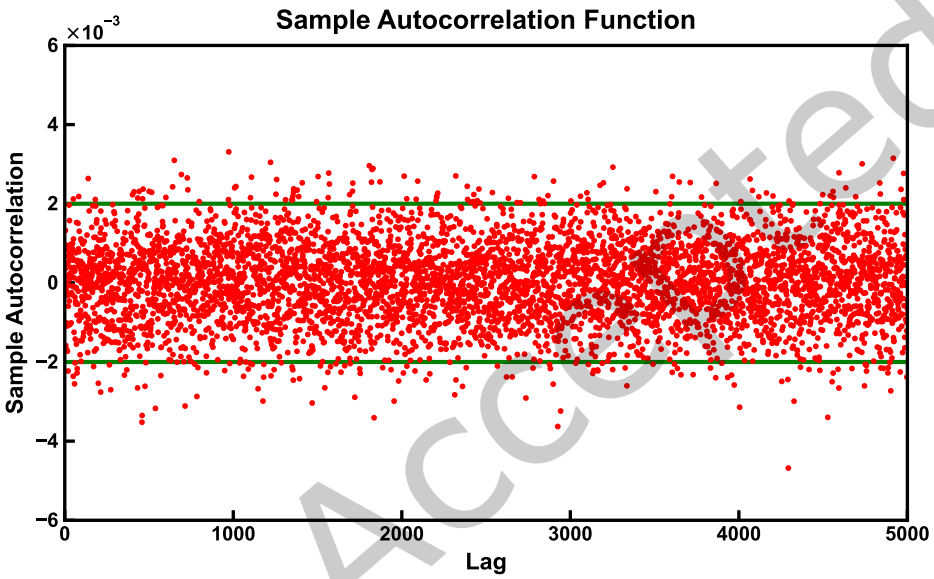

Fig. 15. Sample Autocorrelation Function (ACF) of the PUF responses.

6.3 Reliability

To validate the efficacy of the pre-screening strategy in silicon implementation, comprehensive evaluations were conducted on both 64-stage and 128-stage configurations. The experiments spanned a temperature range from 20°C to 100°C, with the screening threshold rigorously fixed at $T_{HD} = 0.40$. The comparative performance outcomes derived from these tests are presented in Fig. 16.

As illustrated by the red bars, the native reliability suffers from monotonic thermal decay, dropping to 86.48% (128-stage) at 100°C due to intensified thermal noise. In contrast, the proposed screening mechanism (blue bars) demonstrates thermal insensitivity, effectively arresting this decay to sustain a robust reliability of 92.15% even under extreme heat. However, this enhancement involves a trade-off, as indicated by the inverse correlation between the Qualification Rate (green curves) and temperature. Notably, these empirical data and observed trends exhibit a high degree of consistency with the findings from the simulation phase; this alignment between the FPGA experiments and Python simulations fully meets our theoretical expectations.

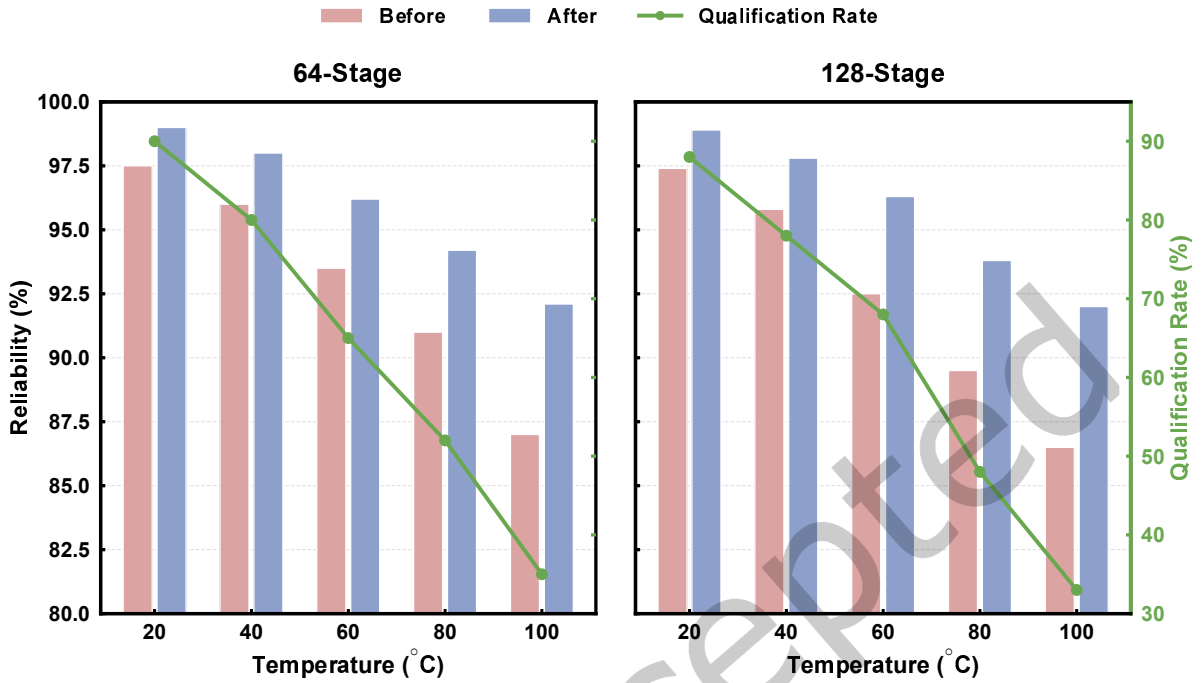

Fig. 16. Reliability performance of 64-stage and 128-stage architectures under varying temperatures (20°C ~ 100°C).

6.4 Modeling Attack Resistance

The attacks were implemented using Python 3.10 with `scikit-learn` and `PyTorch` frameworks. To ensure the evaluation represents a worst-case security threat (i.e., a computationally robust adversary), we rigorously optimized the hyperparameters based on state-of-the-art benchmarks. The detailed configurations are summarized in Table 3.

Key configurations include: Following the comprehensive analysis by Wisiol et al. [26], we adopted the Tanh activation function instead of the traditional ReLU for all neural network architectures (MLP and DNN). As demonstrated in [26], Tanh provides zero-centered outputs that significantly accelerate convergence for parity-based PUF modeling. Furthermore, we adopted the "Improved LR" architecture proposed by Wisiol et al. [26]. This variant is implemented as a single-layer neural network utilizing the Adam optimizer and Tanh activation, which has been proven to significantly enhance convergence robustness compared to standard statistical solvers. For CMA-ES, we strictly adhered to the theoretical guidelines proposed by Hansen [8]. Specifically, the population size λ was dynamically calculated based on the problem dimension n (i.e., stage number) using the formula $\lambda = 4 + \lfloor 3 \ln(n) \rfloor$, ensuring sufficient exploration capability.

Figure 17 illustrates the prediction accuracy of the four modeling attacks against both 64-stage and 128-stage instances, with the training set size scaling from 1×10^5 to 1×10^6 CRPs.

As observed, none of the algorithms succeeded in breaking the security of the proposed PUF. The prediction accuracy for all attack vectors fluctuates stochastically around the ideal value of **50%** (within a negligible margin of ±0.5%). Crucially, the curves exhibit no upward trend as the volume of training data increases. This flatlining behavior indicates that the complex obfuscation logic effectively conceals the internal delays,

Table 3. Hyperparameter Settings for Modeling Attack Algorithms

Parameters	LR (Improved)	CMA-ES	MLP	DNN
Implementation	PyTorch	cma (Python)	PyTorch	PyTorch
Optimizer	**Adam** [26]	-	**Adam** [26]	**Adam**
Learning Rate	10^{-2}	-	10^{-3}	10^{-3}
Loss Function	BCELoss	-	BCELoss	BCELoss
Activation	**Tanh** [26]	-	**Tanh** [26]	**Tanh**
Hidden Layers	None (Linear)	-	**Adaptive** $(n, 2n, n)$	**Fixed** (4×128)
Population Size	-	$4 + \lfloor 3 \ln(n) \rfloor$ [8]	-	-
Batch Size	Full Batch	-	128	256
Max Epochs/Gen.	1000	200	100	100

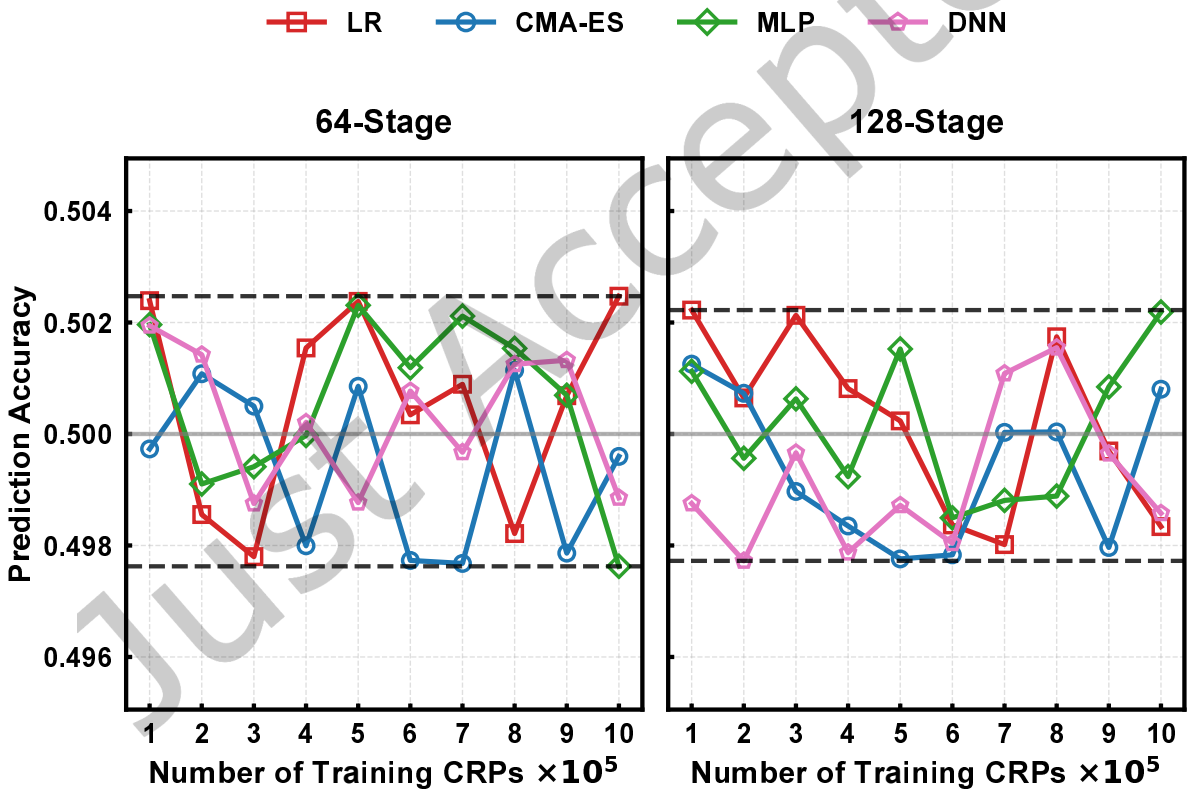

Fig. 17. Prediction accuracy of modeling attacks (LR, CMA-ES, MLP, DNN) versus training set size for 64-stage (left) and 128-stage (right) configurations.

rendering the Challenge-Response behavior computationally unpredictable. Even with 10^6 high-precision CRPs and deep learning models equipped with robust optimization strategies, the adversary cannot extract a statistically significant model better than random guessing. Furthermore, the fact that this resistance persists even on FPGA-generated datasets with inherent physical noise confirms the proposed architecture's security integrity in real-life applications.

6.5 Comparison with State-of-the-Art

To comprehensively evaluate the overall performance of the proposed architecture, Table 4 compares our design with several state-of-the-art Strong PUF schemes recently, covering security resilience, statistical properties, reliability, and hardware resource overhead.

As shown in Table 4, our design has been rigorously tested against a comprehensive set of modeling attacks, including LR, CMA-ES, MLP, and DNN. Even with a training dataset of 1 million CRPs, the prediction accuracy for all attack vectors strictly converges to the ideal value of $\approx 50\%$. By comparison, some schemes, such as the CT PUF [30] or specific configurations of the cSOI PUF [28], exhibit slight information leakage (e.g., accuracy $> 55\%$) under certain attacks, or lack reported data for high-order DNNs. In terms of hardware efficiency, our architecture demonstrates a significant advantage over existing solutions and proves to be exceptionally lightweight [9, 18, 27, 28]. Furthermore, this efficiency is achieved without sacrificing stability, as the reliability is maintained at a high level across varying operational conditions. It is worth noting that most baselines are implemented on identical or architecturally similar Xilinx 7-series FPGA platforms (Artix-7 or Zynq-7000), ensuring a fair comparison baseline regarding hardware complexity.

Table 4. Performance Comparison with State-of-the-Art PUF Architectures

PUF Designs	Stage	MAX Training CRPs	Uniformity (%)	Uniqueness (%)	Reliability (%)	Modeling Accuracy (%)				Hardware Cost	Platform
						LR	CMA-ES	MLP	DNN		
8-Hetero-FFXOR [1]	64	1M	≈50.00	≈50.00	99.45	≈50	≈50	≈50	#	1514 LUTs + 225 FFs	Artix-7
(64,5)-MPUF [18]	64	600K	50.02	49.95	98.67	≈50	≈50	#	#	4736 LUTs + 37 FFs	Simulation
(64,16)-SRPUF [9]	64	500K	50.14	50.03	95.58	49.91	50.20	#	50.28	20624 LUTs + 80 FFs	Artix-7
CT PUF [30]	64	100K	∼50	∼50	> 96	∼60	∼60	#	∼55	731 LUTs + 486 FFs	Zynq-7000
(64, 2)-cSOI PUF [28]	64	40M	49.10	49.80	98.30	#	55.24	#	58.71	763 LUTs + 264 FFs	Artix-7
(64, 4)-cSOI PUF [28]	64	40M	50.00	49.80	97.30	#	49.94	#	50.55	1035 LUTs + 264 FFs	Artix-7
TP-PUF (Bit 5) [27]	32×32	>200M	≈ 50.00	49.34	96.59	#	≈ 50	#	50.43	2277 LUTs + 451 FFs	Artix-7
TP-PUF (Bit 6) [27]	32×32	>200M	≈ 50.00	57.97	98.31	#	≈ 50	#	50.90	2277 LUTs + 451 FFs	Artix-7
SCD-PUF [15]	64	1M	49.57	49.88	97.79	51.28	#	51.29	#	964 LUTs + 361 FFs	Zynq-7000
This Work											
Proposed PUF	**64**	**1M**	**50.18**	**49.91**	**92.14 – 98.26**	**50.25**	**49.96**	**49.76**	**49.89**	**336 LUTs + 288 FFs**	**Artix-7**
Proposed PUF	**128**	**1M**	**49.90**	**49.82**	**92.15 – 98.33**	**49.83**	**50.08**	**50.22**	**49.86**	**592 LUTs + 514 FFs**	**Artix-7**

7 Conclusion

In this work, we presented a lightweight Strong PUF architecture that effectively mitigates machine learning modeling attacks by elevating the physical mapping to a Keyed-PRF paradigm. Our approach leverages a Weak PUF-seeded sponge structure to reconfigure the system logic, instantiating the intrinsic process variations as a hidden key. Furthermore, we establishes a tight structural coupling between the physical delay characteristics and the logical state evolution, transforming the system into a Hidden Markov Model that successfully resists deep learning attacks. Extensive evaluations on 28nm FPGA platforms confirm that the proposed architecture is immune to state-of-the-art modeling attacks, including Deep Neural Networks, with the prediction accuracy remaining at the random-guess level of 50%. The proposed reliability pre-screening strategy further guarantees

robust operation under varying environmental noise. With its low hardware complexity and proven security properties, this design offers a promising root-of-trust solution for resource-constrained IoT devices.

References

[1] S. V. S. Avvaru, Z. Zeng, and K. K. Parhi. 2020. Homogeneous and Heterogeneous Feed-Forward XOR Physical Unclonable Functions. *IEEE Transactions on Information Forensics and Security* 15 (2020), 2485–2498.

[2] C.-H. Chang, Y. Zheng, and L. Zhang. 2017. A retrospective and a look forward: Fifteen years of physical unclonable function advancement. *IEEE Circuits and Systems Magazine* 17, 3 (2017), 32–62.

[3] Y. Cui, J. Li, Y. Chen, C. Wang, C. Gu, M. O'neill, and W. Liu. 2024. An efficient ring oscillator PUF using programmable delay units on FPGA. *ACM Transactions on Design Automation of Electronic Systems* 29, 1 (2024), 1–20.

[4] Y. Cui, C. Wang, W. Liu, Y. Yu, M. O'Neill, and F. Lombardi. 2016. Low-cost configurable ring oscillator PUF with improved uniqueness. In *2016 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 558–561.

[5] J. Gan, H. Luo, Y. Xie, and L. Liang. 2025. A Markov-Chain Based PUF Using Chain-Block-Obfuscation Mechanism Resisting Machine Learning Attacks with High Uniformity Robustness. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2025).

[6] F. Ganji, D. Forte, and J.-P. Seifert. 2019. PUFmeter: a property testing tool for assessing the robustness of physically unclonable functions to machine learning attacks. *IEEE Access* 7 (2019), 122513–122521.

[7] Y. Gao, J. Chilaka, E. Pantoja, R. Klenke, and M. Stan. 2024. CryptoPUF: A Lightweight and ML-Resilient Strong PUF Based on a Weak PUF and Crypto Core. In *2024 IEEE International Conference on RFID Technology and Applications (RFID-TA)*. IEEE, 173–176.

[8] N. Hansen. 2016. The CMA evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772* (2016).

[9] S. Hou, D. Deng, Z. Wang, J. Shi, S. Li, and Y. Guo. 2021. A dynamically configurable LFSR-based PUF design against machine learning attacks. *CCF Transactions on High Performance Computing* 3 (2021), 31–56.

[10] Z. Huang, Y. Lin, Z. Wang, Y. Lu, H. Liang, J. Bian, Y. Ouyang, T. Ni, and X. Wen. 2025. A Parallel Feedback Obfuscation Strong PUF Against Machine-Learning Modeling Attacks and Lightweight Authentication Protocol. *IEEE Transactions on Dependable and Secure Computing* (2025). Early Access.

[11] G. Li, L. Zhao, Z. Zhou, P. Wang, X. Ma, Y. Zhang, and Z. Wang. 2025. Feistel-PUF: Sequential Obfuscation-Based Machine Learning Attack Resistant Physical Unclonable Function for IoT Device Security Authentication. *IEEE Internet of Things Journal* (2025).

[12] D. Lim, J. W. Lee, B. Gassend, G. E. Suh, M. Van Dijk, and S. Devadas. 2005. Extracting secret keys from integrated circuits. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 13, 10 (2005), 1200–1205.

[13] N. Mishra, K. Pratihar, S. Mandal, A. Chakraborty, U. Rührmair, and D. Mukhopadhyay. 2024. Calypso: An enhanced search optimization based framework to model delay-based PUFs. *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2024), 501–526.

[14] P. H. Nguyen, D. P. Sahoo, C. Jin, K. Mahmood, U. Rührmair, and M. Van Dijk. 2019. The Interpose PUF: Secure PUF Design against State-of-the-art Machine Learning Attacks. *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2019), 243–290.

[15] Y. Peng, D. Deng, Z. Wang, and Y. Guo. 2024. SCD-PUF: Shuffled Chaotic-dual-PUF With High Machine Learning Attack Resilience. In *2024 IEEE International Test Conference in Asia (ITC-Asia)*. IEEE, 1–6.

[16] U. Rührmair et al. 2013. PUF modeling attacks on simulated and silicon data. *IEEE Transactions on Information Forensics and Security* 8, 11 (2013), 1876–1891.

[17] U. Rührmair, F. Sehnke, J. Sölter, G. Dror, S. Devadas, and J. Schmidhuber. 2010. Modeling attacks on physical unclonable functions. In *Proceedings of the 17th ACM conference on Computer and communications security (CCS)*. 237–249.

[18] D. P. Sahoo, D. Mukhopadhyay, R. S. Chakraborty, and P. H. Nguyen. 2018. A Multiplexer-Based Arbiter PUF Composition with Enhanced Reliability and Security. *IEEE Trans. Comput.* 67, 3 (2018), 403–417.

[19] P. Santikellur, A. Bhattacharyay, and R. S. Chakraborty. 2019. Deep Learning based Model Building Attacks on Arbiter PUF Compositions. Cryptology ePrint Archive, Report 2019/566.

[20] P. Santikellur and R. S. Chakraborty. 2021. A Computationally Efficient Tensor Regression Network-Based Modeling Attack on XOR Arbiter PUF and Its Variants. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 40, 6 (2021), 1197–1206.

[21] J. Shi, Y. Lu, and J. Zhang. 2020. Approximation attacks on strong PUFs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 10 (2020), 2138–2151.

[22] G. E. Suh and S. Devadas. 2007. Physical Unclonable Functions for Device Authentication and Secret Key Generation. In *2007 44th ACM/IEEE Design Automation Conference*. IEEE, 9–14.

[23] L. Tang, H. Ji, K. Liu, J. Zhou, and W. Fan. 2024. A Hybrid Strong-Weak PUF Against Machine Learning Attacks with High Reliability. In *2024 9th International Conference on Integrated Circuits and Microsystems (ICICM)*. IEEE, 402–406.

[24] W. Trappe, R. Howard, and R. S. Moore. 2015. Low-energy security: Limits and opportunities in the Internet of Things. *IEEE Security & Privacy* 13, 1 (2015), 14–21.

[25] E. I. Vatajelu, G. Di Natale, and P. Prinetto. 2016. Towards a highly reliable SRAM-based PUFs. In *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 273–276.
[26] N. Wisiol, B. Thapaliya, K. T. Mursi, J.-P. Seifert, and Y. Zhuang. 2022. Neural network modeling attacks on Arbiter-PUF-based designs. *IEEE Transactions on Information Forensics and Security* 17 (2022), 2719–2731.
[27] C. Xu, J. Zhang, M.-K. Law, X. Zhao, P.-I. Mak, and R. P. Martins. 2023. Transfer-Path-Based Hardware-Reuse Strong PUF Achieving Modeling Attack Resilience With >200 Million Training CRPs. *IEEE Transactions on Information Forensics and Security* 18 (2023), 2188–2203.
[28] C. Xu, L. Zhang, P.-I. Mak, R. P. Martins, and M.-K. Law. 2024. Fully Symmetrical Obfuscated Interconnection and Weak-PUF-Assisted Challenge Obfuscation Strong PUFs Against Machine-Learning Modeling Attacks. *IEEE Transactions on Information Forensics and Security* 19 (2024), 3927–3942.
[29] X. Xu, H. Liang, Z. Huang, C. Jiang, Y. Ouyang, X. Fang, T. Ni, and M. Yi. 2017. A highly reliable butterfly PUF in SRAM-based FPGAs. *IEICE Electronics Express* 14, 14 (2017), 20170551.
[30] J. Zhang, C. Shen, Z. Guo, Q. Wu, and W. Chang. 2022. CT PUF: Configurable Tristate PUF Against Machine Learning Attacks for IoT Security. *IEEE Internet of Things Journal* 9, 16 (2022), 14452–14462.

Received 6 January 2025; revised 10 February 2026; accepted 1 March 2026